

GrocerEase - Cursor chat log: dev spec automation

Source transcript: /Users/evelynlo/.cursor/projects/Users-evelynlo-Desktop-GrocerEase/agent-

--- Turn 1 (user) ---

ok now i want to do this:

6. Automated development specifications (200 points)?

For each of the three user stories, put in place an LLM-assisted code documentation process that

Whenever a PR connected to a feature that implements a user story is approved, your automation

Turn in:

The prompts used to generate a new development specification. Make use of the prompts you developed

The prompts used to update an existing development specification after the user story or the code

For each story, provide

A URL to the approved pull request that contains the commit(s) and review of the development

A URL to the GitHub Issue that was created to track the creation of the development specification

Attach chat logs or shareable URLs accessible to all instructors.

can you tell me a roadmap on how you want to implement this

--- Turn 2 (assistant) ---

Here's a concise **roadmap** that matches the rubric (automation in the workflow, PR-approved

1. Lock down what "development specification" means (P2 alignment)

- Reuse your **P2 template** (sections, tone, required headings: story, acceptance criteria, APIs,
- Decide **one file per user story** (e.g. `docs/dev-specs/US{n}-{slug}.md`) or one monolith with
- Add a short **docs/dev-specs/README.md** explaining purpose, when specs update, and how they

2. Choose the automation surface (must be "pointable," not only chat)

Recommended default: **GitHub Actions** workflow that runs when a PR is **merged** (proxy for

- **Trigger A (simplest):** `on: pull_request: types: [closed]` + `if:`

- **Trigger B (stricter "approved?"):** `pull_request_review` with `state == approved` **or** use

LLM invocation: a step that calls **OpenAI / Anthropic / etc.** via API key in **GitHub Actions**

3. Two prompt assets (what you turn in for the assignment)

Check in **versioned prompt files** (so the PR that adds automation also shows the prompts):

| Asset | Purpose | Suggested path |

|-----|-----|-----|

| **New spec** | Full doc from story + code diff + PR description | `docs/dev-specs/prompts/new-

spec.md

| **Update spec** | Existing spec + ?what changed? (diff, issue, PR notes) | `docs/dev-

specs/pr

Workflow reads these files and sends them as the **system + user** messages (or a single composed

user mes

4. Workflow behavior (end-to-end)

1. **Detect story** ? From PR: label `story:US5`, or linked issue `Closes #123` where #123 is a US

issue, or

2. **Gather context** ? Checkout merge base + PR head; `git diff`; optional `gh pr view`; linked

issue bo

3. **Call LLM** ? Inject P2 template + repo rules + diff + prompts from ?3.

4. **Write file** ? Update or create the markdown under `docs/dev-specs/`.

5. **Open a follow-up PR** (hygiene) ? Bot branch `docs/spec-update-us5-<sha>`, commit ?docs(spec):

regenera

6. **Human / required review** ? Merge that PR so graders see **review thread** on the spec PR

(meets ?

Alternative: single workflow commits to `main` with PAT?works but weaker hygiene; **dedicated PR**

per spec

5. GitHub hygiene & grading artifacts

- **Issue per story (or per spec update cycle):** e.g. ?Dev spec: US5 ? automated generation from PR

#112?. *

- **Approved PR URL** ? The **spec PR** (or the feature PR if you merge spec in same PR?less ideal)

with visib

- **Chat logs** ? If everything runs in Actions, **workflow run URL** + **job logs** (redact

secrets)

6. Map to your ?three user stories?

- Pick **three concrete stories** (e.g. US5 search, US8 deals, one more you already merged).

- For each: one **tracking issue**, one **feature PR** (already merged), one **spec PR** generated

by autom

- Document in `docs/dev-specs/README.md` the **mapping** story ? issue ? spec file.

7. Safety & quality gates

- **Secrets:** only in `GITHUB\_TOKEN` / org secrets; never echo API keys in logs.

- **Idempotency:** if spec unchanged, skip commit or skip PR.

- **Size limits:** truncate huge diffs; point LLM to key files + `git diff --stat`.

- **Optional:** `markdownlint` or a simple ?required sections? script in CI on `docs/dev-

specs/*.

8. Suggested implementation order (timeline)

Phase	Deliverable
0	P2 template + `docs/dev-specs/` layout + README
1	Prompt files (new + update) checked in
2	Manual dry-run script (local) that prints the composed prompt + calls API once
3	GitHub Action: merged PR ? generate ? branch ? PR
4	Issue templates / labels for `story:USn`
5	Run for 3 stories; collect URLs for turn-in
6	Polish: ?update? path when spec file already exists

9. What to turn in (checklist against rubric)

- **Prompts (new + update)** ? In repo + linked from workflow or README.
- **Per story:** URL of **approved PR** containing spec commit(s) + review; URL of **GitHub Issue**
- **Automation proof** ? Workflow file + run logs (and/or instructor-accessible chat URL).

Pragmatic note: GitHub?s native ?Copilot coding agent? or **third-party bots** can shorten build

If you share where P2 lives in the repo (path) or which three stories you?re standardizing on, the

[REDACTED]

--- Turn 3 (user) ---

here is p2:

Skip to content

CMU 17-356: Software Engineering for Startups

Project 2 Sprint 1

Search

CMU 17-356: Software Engineering for Startups

Home

Overview

Syllabus

Teamwork Practices

Staff

Assignments

Assignments

Homework 1

Homework 2

Recitations

Project

Project

Project 0 Team Contract

Project 1 Product Specification

Project 2 Sprint 1

Project 3 Sprint 2

Project 4 Sprint 3

On This Page

Submission

Learning Goals

1. Project Context

2. User Stories (100 points)
3. User Interfaces (100 points)
4. Implementation (100 points)

Project 2: Sprint 1?

This project assignment is worth 300 points.

This homework is to be done as a team.

P2 is due on Gradescope Thursday, March 26, 2026 11:59pm.

Submission?

Submit P2 as a PDF file to Gradescope.

If you prepare the response in some other software (like Tex), please export as PDF before

Learning Goals?

Design and implement the front-end user interface of your startup's application.

Use a Large Language Model (LLM) to generate software artifacts, including user stories, UI

1. Project Context?

In modern software engineering workflows, LLMs can generate code and interfaces very quickly.

In this project assignment, you will:

Use an LLM to generate the user stories describing your product's application.

Use the LLM to generate the UI and first version of the code.

2. User Stories (100 points)?

In this section, you will create user stories that correspond to the primary use cases of the

The user stories should be customized by your understanding of the primary use cases of your

The total user stories your team should write is 2 times the number of team members. If your team

Write the computed number of user stories. Make sure each user story uses the proper template:

As a <user>, I want <action> so that <benefit>. Then add your machine and human acceptance criteria

Add a one paragraph explanation of why each user story is important to the overall user experience

Make sure each user story can be designed, implemented, tested, and verified in under 2 weeks. If it

Evaluate the user stories.

Use the INVEST framework we learned in class to evaluate each user story.

Identify the 3 user stories that you think are most critical to the business success of the

Identify the 3 user stories that you think will be easiest to implement and explain why.

Identify the 3 user stories that you think will be easiest to test via machine-executable tests and

Identify the 3 user stories that you think will be easiest for human to decide that it was

Prioritize the user stories in order that you should implement them. Justify your implementation

Identify and label the "core" user story that creates the base functionality of your application.

Create GitHub Issues for each user story (along with its acceptance criteria) in your team's GitHub

If you used an LLM for any part of this section, please turn in a copy of the chat log or a URL to

3. User Interfaces (100 points)?

Create high-fidelity user interfaces for each of your user stories using an AI tool.

Include all states of the interface: e.g., empty state, loading, error, success, etc.

Write up the rationale for major design choices (e.g., navigation flow, component reuse,

For each element in the UI, identify which user story(ies) required it. If it cannot be connected to

Be sure that the design of each user interface is compatible with all of the others, meaning it

Check in high-resolution screenshots of your user interface prototypes into your GitHub repository

If you used an LLM for any part of this section, please turn in a copy of the chat log or a URL to

4. Implementation (100 points)?

Implement the user stories in code for both the frontend and backend using a popular framework of

For each user story, create a new branch in Git. Write appropriate code for this user story. Update

Your implementation must:

Include responsive design for at least two screen sizes.

Ensure accessibility compliance with WCAG 2.2 and alt-text for images where appropriate.

The core user story should be checked into your GitHub repository first.

Check in the code for each user story in its feature branch.

Do a code review on the PR and iterate until your teammates are satisfied with the quality of the

Approve the PR and merge it into main. Remember to keep the status of the user story up to date on

Turn in a URL to each user story's PR.

Screen record yourself showing off all of the user stories implemented in your app to a customer in

Best practice tip: Invent a scenario that motivates a set (usually some, but not all) of user

Upload the screen recording to YouTube (set to Unlisted and provide the URL).

If you used an LLM for any part of this section, please turn in a copy of the chat log or a URL to

Back to top

Made with Material for MkDocs

ya so whats the workflow now, i need to start to finish

--- Turn 4 (assistant) ---

Here?s a **start-to-finish workflow** that lines up **P2** (what you built and how you described it)

A. What you already satisfied with P2 (Gradescope)

- **P2 deliverable** = **PDF** (user stories, UI rationale, implementation notes as your team wrote it). That?
- Anything below is **ongoing engineering + later assignment(s)** (e.g. P3 ?6), not a second P2 PDF.

B. Single source of truth in GitHub (do this first)

1. **Issues**

One issue per user story (or per ?spec refresh?) with: title `US{n}: ?`, acceptance criteria in

body, lab

2. **Branches**

`feature/us-{n}-short-slug` from **up-to-date `main`**.

3. **PRs**

- Description: what changed, **Closes #issue?** for the story (or ?Refs? if split).
- **Review + approve + merge** (secondary owner comment = satisfies P2 implementation bullet if you

still need

4. **Optional but helpful**

Label merged PRs with **story:US5** (etc.) so automation knows which spec file to update.

C. ?Development specification? in the repo (what automation maintains)

For **each of the 3 stories** you'll highlight for the assignment:

| Piece | Purpose |

|-----|-----|

| `docs/dev-specs/US{n}-?.md` | Living spec: story, AC, main flows, APIs/routes, DB, UI entry

points, e

| `docs/dev-specs/prompts/spec-new.md` | Prompt template for **first-time** generation. |

| `docs/dev-specs/prompts/spec-update.md` | Prompt template when the file **already exists** and

code/issu

| `.github/workflows/dev-spec.yml` | **Automation** (not chat-only). |

Content of the spec should **mirror P2 intent** (same story + AC + where UI lives + implementation

reality) b

D. End-to-end flow (every feature PR)

```text

1. Open feature branch from main
2. Implement + tests; open PR ? review ? approve ? merge to main
3. (Automation) On merge (or on ?approved? if you configure that):
  - Collect: diff, PR body, linked issue(s), existing docs/dev-specs/US{n}\*.md if any
  - Call LLM with spec-new OR spec-update prompt + P2-style checklist embedded in prompt
4. Automation commits to a new branch, opens a **docs-only PR** (e.g. docs/spec-us5-from-pr-123)
5. Human quickly reviews spec PR ? approve ? merge
6. Tracking: **Issue** ?Automated dev spec for US5 (PR #123)? linked in that PR/commit body

```

Why a second PR for the spec? Clean history, visible **review of the spec itself**, easy URL for

graders (

E. What you turn in (assignment checklist)

For **each of the 3 user stories**:

| Deliverable | Where it comes from |

|-----|-----|

Prompt for **new** spec	`docs/dev-specs/prompts/spec-new.md` (+ link in README)
Prompt for **update** spec	`docs/dev-specs/prompts/spec-update.md`
URL: **approved PR** that merged the spec (with review)	The **docs/spec-?** PR
URL: **GitHub Issue** tracking spec work	Linked from PR description / commit (`Fixes #?` / `Refs

#?`) |

| Chat / logs | **GitHub Actions run URL** (preferred) or instructor-accessible chat export |

For **each story**, also have (or reference) the **feature implementation PR** URL if the rubric

asks for

F. Practical order (this week ? done)

| Step | Task |

|-----|-----|

- | 1 | Create `docs/dev-specs/` + README + empty or stub `US{n}.md` for 3 stories. |
- | 2 | Add `spec-new.md` and `spec-update.md` (your real prompts; version in Git). |
- | 3 | Add GitHub Action + secrets (`OPENAI\_API\_KEY` or chosen provider). |
- | 4 | Dry-run on **one** merged PR (manually re-run workflow or test branch). |
- | 5 | Fix prompt until output matches your P2-style sections. |
- | 6 | Run for **all 3** stories; open/merge **3 spec PRs**; open **3 tracking issues**. |
- | 7 | Collect URLs + export or link Action logs for Gradescope/README for instructors. |

G. How P2 fits the prompts (without resubmitting P2)

In both prompts, include a fixed block like:

- ?Follow this outline: User story (As a?), Acceptance criteria, UI surfaces (screenshots path in

repo), Im

Paste **short excerpts** from your P2 PDF (the 3 chosen stories only) as **authoritative intent**;

let the LL

Bottom line: Your **workflow** is: **feature PR merges** ? bot opens spec PR from prompts ? human

approves

If you want the next concrete step in-repo, say whether you're standardizing on **GitHub Actions** +

OpenAI*

[REDACTED]

--- Turn 5 (user) ---

ya so i just need to create the bot that gets triggered when a pr gets approved? im still confused

what this

--- Turn 6 (assistant) ---

You're not being asked to build a general-purpose ?bot.? You're being asked to show **repeatable**

automati

What ?the assignment? is really checking

1. **There is a defined artifact** ? a **development specification** (structured doc: story, behavior,
2. **An LLM produces or updates that artifact** ? not you typing the whole file by hand every time.
3. **That happens from your dev workflow** ? something in CI/CD (usually **GitHub Actions**) runs

main cod

- **On merge to `main`** (simplest, still ?after review? if you require review before merge), or
- **On `pull\_request\_review`** with **approved`** (stricter; a bit more fiddly with bots and

Either can be defensible if you **say in your write-up** what you chose and why (merge = team

4. **Git hygiene** ? the spec change should show up as **real Git history**: e.g. a **branch + PR**

5. **Evidence of the LLM** ? the **prompts** checked in (new vs update), plus **logs** or a shareable

So: **trigger** is only one piece. The full picture is: **trigger** ? gather context (diff, issue, old

Do you ?just? create a bot on approval?

Almost: you create **automation** (typically a **GitHub Actions workflow**) that:

- fires on your chosen event (approval or merge),
- calls the LLM API,
- updates/creates the spec file,
- opens a PR (or commits in a controlled way),
- you link an **issue** and show **review** on that PR.

If you only had a webhook that DMs you and never commits, that would **not** match ?check it into

One sentence you can use in your write-up

?We use GitHub Actions on [merge to main | PR approval] to regenerate `docs/dev-specs/USn-?.md` via

That?s the clearest story for graders **and** it matches what they?re grading. If you tell me

[REDACTED]

--- Turn 7 (user) ---

ok then create a new branch and implement this? you probably need a yml file and is it possibel to

--- Turn 8 (assistant) ---

Implementing a GitHub Actions workflow and supporting scripts. Checking whether GitHub Models works

[REDACTED]

--- Turn 9 (assistant) ---

[REDACTED]

--- Turn 10 (assistant) ---

[REDACTED]

--- Turn 11 (assistant) ---

Implementing automation using GitHub's official `actions/ai-inference` action (GitHub Models +

[REDACTED]

--- Turn 12 (assistant) ---

[REDACTED]

--- Turn 13 (assistant) ---

[REDACTED]

--- Turn 14 (assistant) ---

Fixing the prompt template and adding the workflow. Correcting the commit step: preserve the

[REDACTED]

--- Turn 15 (assistant) ---

[REDACTED]

--- Turn 16 (assistant) ---

[REDACTED]

--- Turn 17 (assistant) ---

[REDACTED]

--- Turn 18 (assistant) ---

[REDACTED]

--- Turn 19 (assistant) ---

[REDACTED]

--- Turn 20 (assistant) ---

[REDACTED]

--- Turn 21 (assistant) ---

[REDACTED]

--- Turn 22 (assistant) ---

[REDACTED]

--- Turn 23 (assistant) ---

Here's what's in place and how it relates to GitHub's built-in AI.

Can you use only GitHub's AI?

- In GitHub Actions: Yes, via GitHub Models(<https://github.com/marketplace/models>) and the

- That is not the same as Copilot Chat in the IDE. Copilot is mainly interactive; this path is

If Models aren't enabled for the org/account, the job will fail until that's turned on; docs/dev-

What was implemented (branch 'feature/dev-spec-automation', commit '549e434')

| Piece | Role |

|-----|-----|

| .github/workflows/dev-spec-on-merge.yml | After a PR is merged into 'main', if it has label

| .github/prompts/dev-spec-generate.prompt.yml | Template for GitHub Models ('openai/gpt-4o';

| docs/dev-specs/prompts/spec-new.md | New spec instructions (for Gradescope / audit). |

| docs/dev-specs/prompts/spec-update.md | Update spec instructions. |

| docs/dev-specs/README.md | How to opt in ('dev-spec:' label), hygiene, links. |

How you use it

1. Add label 'dev-spec:US5' on the feature PR (slug = part after ':' ? file docs/dev-

2. Get PR **reviewed and merged** to ``main`` (your approval gate).
3. Open the **Actions** run, then the **bot-opened docs PR**; add **``Refs #issue``** on that PR if
4. **Approve and merge** the docs PR ? that's the URL you can submit as ?PR with review of the

Push the branch

```
``bash
git push -u origin feature/dev-spec-automation
``
```

Then open a PR to ``main`` so the workflow exists on the default branch (workflows only run from the

Note: We used **merge** to ``main`` as the trigger (reliable, matches ?after review?). If the

[REDACTED]

--- Turn 24 (user) ---

i think youre suppose to trigger it after a pr gets approved

--- Turn 25 (assistant) ---

Updating the workflow to run on ``pull_request_review`` when the review state is ``approved``, and

[REDACTED]

--- Turn 26 (assistant) ---

Switching the trigger to ``pull_request_review`` (approved) and loading the existing spec from

[REDACTED]

--- Turn 27 (assistant) ---

[REDACTED]

--- Turn 28 (assistant) ---

[REDACTED]

--- Turn 29 (assistant) ---

[REDACTED]

--- Turn 30 (assistant) ---

Here's what changed:

Trigger (what you asked for)

- **Event:** ``pull_request_review`` with ``types: [submitted]``
- **Job runs only when:**
- ``github.event.review.state == 'approved'``
- PR **base** is ``main``
- PR is still **open**

So it fires when someone hits **Approve** on GitHub, not when the PR is merged.

File rename

- **Removed:** ``github/workflows/dev-spec-on-merge.yml``
- **Added:** ``github/workflows/dev-spec-on-pr-approval.yml`` (git recorded it as a rename)

Behavior tweak (important for ?approved but not merged?)

- The **previous** spec is loaded with ``git show origin/main:<path>`` so **spec-update** uses what's
- The **diff** is still ``base.sha...head.sha`` from the PR event.

Docs

- `docs/dev-specs/README.md` now describes **approval** as the trigger and that the feature PR can

still be op

Commit

- On branch `feature/dev-spec-automation`: `f816fc9`

Caveats (for your write-up):

- Each **new** approval can start another run (new docs branch includes `run\_id`). If you want ?only

once per

- Reviews from **bots** count as `approved` unless you add a filter like

`github.e

Push when ready: `git push origin feature/dev-spec-automation`

[REDACTED]

--- Turn 31 (user) ---

wait so what does this do,

--- Turn 32 (assistant) ---

It sets up **automatic dev-spec generation** when a PR gets approved.

In plain terms

When someone approves a PR to `main`, this workflow can generate/update a markdown spec file (using

GitHub?

Exact flow

1. Reviewer clicks **Approve** on a PR targeting `main`.

2. Workflow checks for a label like **dev-spec:US5**.

- If missing, it skips.

3. It decides target file from that label:

- `dev-spec:US5` ? `docs/dev-specs/US5-development-spec.md`

4. It gathers:

- PR title/body

- PR diff (`base...head`)

- existing spec from `origin/main` (if any)

- prompt template(s)

5. It calls **actions/ai-inference** (GitHub Models, `GITHUB\_TOKEN`, `models: read`) to generate

markdow

6. It creates a new branch, commits that spec file, and opens a docs PR:

- title like `docs(spec): US5 from approved PR #123`

Why this is useful for your assignment

- It's **automation in workflow** (not chat-only).

- It uses an **LLM** to generate/update spec.

- It checks changes into GitHub with **PR + review trail**.

- You already have separate prompt files for **new** and **update** cases.

If you want, I can also add a one-line guard so it only runs for **human approvals** (ignores bot

approval

--- Turn 33 (user) ---

where would u see the labels

--- Turn 34 (assistant) ---

You see them on the **pull request page on GitHub**, not in the repo files by default.

On GitHub (web)

1. Open the PR (e.g. <https://github.com/gsha22/GrocerEase/pull/123>).

2. On the right sidebar, find **Labels**.

3. Click the gear **?** (or **Labels** if it's expanded) to add/remove labels.

4. Add one that matches what the workflow expects, e.g. `**`dev-spec:US5`**` (exact prefix ``dev-`

`spec:``).

If the sidebar is narrow, scroll the right column; `**Labels**` sits with `**Reviewers**`,

`**Assign`

In GitHub CLI

```
``bash
```

```
gh pr view 123 --json labels
```

```
``
```

In the workflow

The workflow reads labels from the PR payload (``github.event.pull_request.labels``). It does `**not**`

read lab

`**Note:**` Only people with permission to edit labels on the repo can set them; if you don't see the

gear, you

[REDACTED]

--- Turn 35 (user) ---

other prs don't have labels, this is an example of a pr for a user story:

Summary

Closes #5: Story 4 (Filter Stores by Specialty) ? shoppers can filter the store directory by

specialty

GET `/api/stores?category=halal,organic` filters published stores using AND logic via Prisma has on

the categ

StoreFilterBar component with 5 toggleable filter chips ? active filters are visually highlighted

(green) a

StoreDirectory client component wires filter state to API fetching with `useTransition` for snappy,

non-bloc

StoreCard presentational component with category-aware hero emoji and tag badges

Home page replaced static placeholder cards/chips with live `<StoreDirectory />` backed by real data

Seed script with 6 sample Pittsburgh stores covering all category combinations

Categories stored as lowercase strings in the `categories` array field on each Store row ? no join

tables ne

Acceptance Criteria

Filter options include: Asian groceries, halal, organic, produce, EBT accepted

Selecting a filter updates the store list immediately without a full page reload

Multiple filters can be selected simultaneously (AND logic)

Active filters are visually highlighted and individually clearable

GET `/api/stores?category=halal,organic` returns only stores matching all selected tags

Filter labels are clear and culturally respectful

Filtering feels snappy ? results update in under one second

Build passes (next build succeeds with zero errors)

Files Changed

Area Files

API `app/api/stores/route.ts`

Pages `app/(public)/page.tsx`

Components `components/StoreFilterBar.tsx`, `components/StoreDirectory.tsx`, `components/StoreCard.tsx`

Database `prisma/seed.ts`

Config `package.json` (added `db:seed` script, `prisma.seed` config, `tsx dev` dep)

--- Turn 36 (assistant) ---

Perfect example ? with that PR format, you `**don't` need labels`**`.

Your workflow can extract the story directly from PR title/body, e.g.:

- ``Closes #5: Story 4 ...``

- or `Story 4` anywhere in summary

Then map it to:

- `US4` ? `docs/dev-specs/US4-development-spec.md`

Recommended change

Replace `require dev-spec:\*` label? with:

1. Read PR title + body

2. Regex for:

- `Story\s+(\d+)`

- optional `US\s\*(\d+)`

3. Build slug `US<number>`

4. Continue generation as before

Why this is better for your team

- Matches your existing PR habit (`Closes #5: Story 4 ...`)

- No extra manual label step

- Still fully automated and auditable

If you want, I can patch the workflow now so it:

- supports both formats (`Story 4`, `US4`)

- falls back to label only if present

- fails gracefully with a clear log if no story number is found.

[REDACTED]

--- Turn 37 (user) ---

ya patch the workflow

--- Turn 38 (assistant) ---

Patching it now to auto-detect story number from PR title/body (`Story 4` / `US4` / `Closes #5:

Story 4`)

[REDACTED]

--- Turn 39 (assistant) ---

[REDACTED]

--- Turn 40 (assistant) ---

[REDACTED]

--- Turn 41 (assistant) ---

Patched. It now works without labels by default.

What changed

- **Workflow slug detection** in `.github/workflows/dev-spec-on-pr-approval.yml` now:

1. Parses PR **title/body** for `Story N` or `USN` (case-insensitive, supports formats like `Story

4`, `story

2. Falls back to label `dev-spec:US4` only if no story marker is found.

- **Skip message** updated to reflect the new rule:

- skips only when neither story marker nor `dev-spec:\*` label is present.

- **README update** in `docs/dev-specs/README.md`:

- trigger steps now say use PR text marker first, label as optional fallback.

Resulting behavior

For your example PR body (`Closes #5: Story 4 ...`), it will auto-map to:

- `slug = US4`

- spec path: `docs/dev-specs/US4-development-spec.md`

No label needed.

If you want, I can commit these two file changes now.

--- Turn 42 (user) ---

so can the ai detect the pr's summary now for user story and for documentation? how do i test this

--- Turn 43 (assistant) ---

Yes ? with the patch, it now detects the story from PR text (title/body), so it can use your PR

summary

What it detects now

It looks for these patterns in **PR title + body**:

- `Story 4`
- `story: 4`
- `US4`
- `US 4`

If it finds one, it maps to:

- `US4` ? `docs/dev-specs/US4-development-spec.md`

If not found, it falls back to label `dev-spec:US4`.

How to test (end-to-end)

1) First requirement

Make sure this automation PR is merged to `main` first ? workflows only run from the default branch

config.

2) Create a test feature PR

Open any small PR to `main` with title/body containing one marker, e.g.:

- Title: `Story 4: test spec automation`
- Body includes: `Closes #5: Story 4 ...`

3) Trigger approval

Have someone submit an **Approve** review on that PR.

4) Verify workflow run

Go to **Actions** ? `Dev specification (on PR approval)` and open the latest run.

Check logs for:

- resolved slug (should be `US4`)
- generated spec file path
- ?opened docs PR?

5) Verify output

You should see a new bot PR:

- title like `docs(spec): US4 from approved PR #...`
- file changed: `docs/dev-specs/US4-development-spec.md`

Quick failure checks

If it didn't run:

- PR target isn't `main`
- review state wasn't `approved`
- workflow file not on `main` yet

If it ran but skipped:

- no `Story N`/`USN` marker in title/body
- and no `dev-spec:\*` fallback label

If it failed at inference:

- GitHub Models not enabled for repo/org
- missing `models: read` permission (already set in workflow)

If you want, I can also add a temporary `workflow\_dispatch` trigger so you can run it manually for

[REDACTED]

--- Turn 44 (user) ---

what do u mean fall back, it doesnt have labels though: If not found, it falls back to label dev-

--- Turn 45 (assistant) ---

?Fallback? just means **backup option**.

Your workflow now does this in order:

1. Try to detect story number from PR **title/body** (`Story 4`, `US4`, etc.)
2. **Only if that fails**, try label `dev-spec:US4`
3. If both missing, skip

So if your PR text already has `Story 4` (like your example), it **won't** need labels at all.

The label logic is there only to avoid breakage on oddly formatted PRs.

If you want, I can remove label fallback entirely and make it **text-only**.

--- Turn 46 (user) ---

also so it should make a document that says it is for a certain user story, probably link to the

--- Turn 47 (assistant) ---

Yes ? that's exactly what the generated spec should include, and you can make it deterministic.

What the doc should contain (minimum)

For each generated `docs/dev-specs/USN-development-spec.md`:

- **User story ID** (e.g. `US4`) + story statement
- **Linked issue(s)** (e.g. `Closes #5` ? full URL)
- **PR reference** (PR number/title/link)
- **Files changed** grouped by area (API, UI, DB, tests, config)
- **Behavior shipped** (user-visible summary)
- **Acceptance criteria mapping** (which AC is covered / not covered)
- **API/data changes** (routes, models, migrations)
- **Tests/evidence** (test files or scripts)
- **Known gaps / follow-ups**

That will satisfy both engineering usefulness and assignment traceability.

Can it find issue links if not explicitly provided?

Yes, mostly. Three levels:

1. **Best:** parse PR body for `Closes #123`, `Fixes #123`, `Refs #123`
2. **Also good:** use PR linked issues via GitHub API (`closingIssuesReferences`)
3. **Fallback:** if none found, write `No linked issue found in PR metadata`.

So even if someone forgets to write `Closes #...`, you can still often discover linked issues from

What you should add next (small upgrade)

In the workflow ?prepare context? step, append:

- repo owner/name

- PR URL
- extracted issue URLs (from body regex + API if possible)
- diffstat + changed file list (`git diff --name-only`)

Then your prompt can require:

> ?Always include a **Traceability** section with `User story`, `Issue URL(s)`, `PR URL`, and

`Change

If you want, I can patch the workflow + prompt to guarantee those sections appear every run.

[REDACTED]

--- Turn 48 (user) ---

ya patch workflow and guarantee those sections appear

--- Turn 49 (assistant) ---

I'll patch both the workflow and prompts so the generated spec always includes required sections,

and add

[REDACTED]

--- Turn 50 (assistant) ---

[REDACTED]

--- Turn 51 (assistant) ---

Now I'm enforcing exact required headings in prompts and adding a workflow validation gate so the

run fails

--- Turn 52 (assistant) ---

[REDACTED]

--- Turn 53 (assistant) ---

[REDACTED]

--- Turn 54 (assistant) ---

[REDACTED]

--- Turn 55 (assistant) ---

[REDACTED]

--- Turn 56 (assistant) ---

Done ? patched to guarantee those sections and include story/issue/file-change traceability.

What I changed

- **Workflow:** `.github/workflows/dev-spec-on-pr-approval.yml`
- Adds richer context files before LLM call:
- `story\_slug.md` (e.g. `US4`)
- `issue\_links.md` (parsed from PR body refs like `Closes #123`, `Fixes #123`, `Refs #123`)
- `changed\_files.md` (from `git diff --name-only base...head`)
- Passes those into the prompt (`file\_input`).
- Adds a **validation gate** after generation:
- fails the run if any required heading is missing:
- **## User story`**
- **## Summary`**
- **## Scope`**
- **## User-visible behavior`**
- **## Files changed`**
- **## Implementation map`**
- **## Data & persistence`**
- **## APIs`**

- `## Errors & edge cases`
- `## Testing`
- `## Traceability`
- `## Out of scope / follow-ups`
- **Prompt template:** `.github/prompts/dev-spec-generate.prompt.yml`
- now explicitly injects:
 - detected story slug
 - extracted issue links
 - changed files list
- **Prompt rules (new/update):**
 - `docs/dev-specs/prompts/spec-new.md`
 - `docs/dev-specs/prompts/spec-update.md`
- now require exact heading names and force traceability/file grouping behavior.
- **Docs:** `docs/dev-specs/README.md`
- explains required-section validation and issue-link extraction behavior.

If you want, I can commit these changes now.

[REDACTED]

--- Turn 57 (user) ---

ok sure

--- Turn 58 (assistant) ---

Committing these workflow/prompt validation updates on the current branch now.

[REDACTED]

--- Turn 59 (assistant) ---

Committed on `feature/dev-spec-automation`:

- **Commit:** `b70d69d`
- **Message:** `ci(docs): enforce required dev-spec sections and traceability context`
- Working tree is clean.

If you want, I can push this branch and draft a PR description next.

--- Turn 60 (user) ---

wait i already have a pr for this branch

--- Turn 61 (assistant) ---

Perfect ? then you're good. Since the new commits are on the same branch, they'll appear in that

What to do now

- If not pushed yet:
 - `git push`
- Open your existing PR and confirm:
 - `.github/workflows/dev-spec-on-pr-approval.yml`
 - `.github/prompts/dev-spec-generate.prompt.yml`
 - `docs/dev-specs/README.md`
 - `docs/dev-specs/prompts/spec-new.md`
 - `docs/dev-specs/prompts/spec-update.md`

Quick PR note you can add

- ?Updated workflow to trigger on PR approval, infer story from PR summary (`Story N`/`USN`), inject

If you want, I can run `git log -n 3 --oneline` and give you a copy-paste changelog block for the PR

existing PR

issue link

conversa

--- Turn 62 (user) ---

here are the dev specs that are needed:

2. Development Specification (300 points)?

Choose 3 user stories from P2 and ask an LLM document them into 3 separate dev specs. Each dev spec

The primary and secondary owners of the user story.

The date the code was merged into the main branch.

An architecture diagram in Mermaid that also illustrates where the components of the application

An information flow diagram in Mermaid that shows which user information and application data moves

A class diagram in Mermaid that shows the classes relevant to the user story's implementation in

A list of all of the classes in the implementation relevant to this user story. Each class must

A complete list of technologies/libraries/APIs used in your system that you aren't writing yourself.

Indicate what the technology is being used for.

Indicate why that technology was picked over others.

Provide a URL to source location, author, and documentation for each technology.

Don't leave out technologies like language, common libraries, or necessary tools.

Do mention the required version number for each technology.

A list of each data type that you will be storing in a database (i.e. long-term storage). Explain

A list of the user-visible and internally-visible effects if your frontend application:

crashed its process

lost all its runtime state

erased all stored data

noticed that some data in the database appeared corrupt

remote procedure call failed

client overloaded

client out of RAM

database out of space

lost its network connectivity

lost access to its database

bot signs up and spams users

A list all Personally Identifying Information (PII) stored in long-term storage in your system. This

Justify why you need to keep each data item in storage.

How exactly is it stored?

How did the data enter your system?

Through what modules, components, classes, methods, and fields did it move before entering long-term

Through what modules, components, classes, methods, and fields will it move after leaving long-term

List the people on your team who have responsibility for securing each unit of long-term

Describe your procedures for auditing routine and non-routine access to the PII data.

Is the PII of a minor under the age of 18 solicited or stored by the system?

Why?

Does your application solicit a guardian's permission to have that PII?

What is your team's policy for ensuring that minors' PII is not accessible by anyone convicted or

please make it exactly like this

--- Turn 63 (assistant) ---

Updating prompts, the LLM template, and workflow validation so generated dev specs match the

[REDACTED]

--- Turn 64 (assistant) ---

[REDACTED]

--- Turn 65 (assistant) ---

Rewriting prompts to match the course rubric exactly, fixing the issue-link regex, enriching prep

[REDACTED]

--- Turn 66 (assistant) ---

Cleaning up `spec-new.md` (clearer heading list), fixing the issue-ref regex, adding

[REDACTED]

--- Turn 67 (assistant) ---

[REDACTED]

--- Turn 68 (assistant) ---

[REDACTED]

--- Turn 69 (assistant) ---

[REDACTED]

--- Turn 70 (assistant) ---

Here's what was updated to match the **Development Specification (300 pts)** rubric as closely as

1. `docs/dev-specs/prompts/spec-new.md`

Replaced the old lightweight outline with the **exact 11` sections`** your rubric lists, plus the

- **Primary and secondary owners`**
- **Merge date`**
- **Architecture diagram (Mermaid)`**
- **Information flow diagram (Mermaid)`**
- **Class diagram (Mermaid)`**
- **Classes in implementation` (public then private, grouped by concept)**
- **Technologies, libraries, and external APIs` (version, why, URLs, ?)**
- **Database data types and storage` (bytes per field)**
- **Failure mode effects (frontend)` (all listed scenarios)**
- **Personally Identifying Information (PII)` (full sub-questions)**
- **Minors and PII` (all sub-questions)**

No extra top-level **sections are allowed.**

2. `docs/dev-specs/prompts/spec-update.md`

Same rubric headings must exist after an update; merge of new knowledge + revision note under

3. `.github/workflows/dev-spec-on-pr-approval.yml``

- **Fixed** the `Closes #123`` regex (it had escaped ``b`` wrong before).
- Writes **owners_context.md`** (author + assignees) and **merge\_context.md`** (`merged``,

- **Validation** now checks for the **H1** prefix ``# Development specification:`` and **each exact**

4. ``.github/prompts/dev-spec-generate.prompt.yml``

- Injects ``owners_context`` + ``merge_context``.
- Clarifies changed files are **evidence only** (rubric has no `?Files changed?` section).
- ``maxCompletionTokens: 32000`` so long specs are less likely to truncate mid-section.

5. ``.docs/dev-specs/README.md``

- Documents rubric alignment and the **merge-date caveat** (approval usually happens **before**

Commit

``8fb4561`` on ``feature/dev-spec-automation`` ? push to update your existing PR.

Important caveat (merge date):

The rubric wants **date the code was merged into main.** This workflow still runs on **approval**,

If you want the merge date to be **fully automatic**, the next step is a **second workflow** on

[REDACTED]

--- Turn 71 (user) ---

ok tell me what you have done so far and what would happen after a pr gets approved

--- Turn 72 (assistant) ---

What's been done so far (in the repo)

Branch: ``feature/dev-spec-automation`` (with commits including the latest rubric-aligned work,

1. GitHub Actions workflow ? ``.github/workflows/dev-spec-on-pr-approval.yml``

- **Trigger:** ``pull_request_review`` when someone submits a review with ``state == approved``, and

- **Story slug:** Parses PR **title + body** for ``Story N`` or ``USN`` ? e.g. ``US4``. If nothing

- **Context built for the LLM:**

- PR title/body
- Extracted issue URLs from body (``Closes`` / ``Fixes`` / ``Refs`` style)
- PR author + assignees (for primary/secondary owners)
- Merge state: ``merged``, ``merged_at``, plus review ``submitted_at`` (for merge-date wording)
- List of changed files + truncated diff
- Existing spec from ``origin/main`` if that ``.docs/dev-specs/USn-development-spec.md`` already exists

2. LLM call ? ``.github/prompts/dev-spec-generate.prompt.yml`` + ``actions/ai-inference@v1``

- Uses **GitHub Models** (permissions: models: read, default token).
- Large token budget (``maxCompletionTokens: 32000``) so long rubric docs are less likely to cut off.

3. Prompts ? ``.docs/dev-specs/prompts/spec-new.md`` and ``spec-update.md``

- Tell the model to output the **exact CMU-style rubric sections** (owners, merge date, 3? Mermaid

- No extra top-level `###`` sections.

4. Validation gate (same workflow)

- After generation, ``grep`` checks:

- First heading line matches `# Development specification`
- Every required `## ...` heading exists **exactly** (same line text).
- If anything is missing ? **workflow fails** and **no docs PR** is opened.

5. Docs ? `docs/dev-specs/README.md`

- Explains trigger, story detection, validation, and the **merge-date caveat** (approval is usually

What happens after a PR is approved (step by step)

1. GitHub fires **pull_request_review** with `approved`.
2. If the PR is **into `main`**, **open**, and a **story slug** is found (`Story N` / `USN` or `dev-`
3. It checks out the repo, builds the context files + diff, calls **GitHub Models** with your
4. It writes the generated markdown to `docs/dev-specs/USn-development-spec.md` in the workspace.
5. **Validation:** if all required headings + H1 are present ? continue; else **fail**.
6. If validation passes, it creates a **new branch** off `main`, commits only that spec file,
7. The **feature PR** is still open unless your team merges it separately; the generated doc is

Merge date in the doc: On approval, `merged\_at` is usually **null**, so the prompt tells the

Reminder: This only runs on GitHub once the workflow file is on **main** (merge the automation

If you want, I can paste the exact list of validated `##` headings from the workflow file for your

[REDACTED]

--- **Turn 73 (user)** ---

what is: The prompts used to generate a new development specification. Make use of the prompts you

--- **Turn 74 (assistant)** ---

That line is asking you to **submit the actual text you feed the LLM** when you ask it to create a

What counts as the prompts?

Usually it's **two layers**:

1. **Your narrative / structure prompt** ? the rules that say *what sections must exist*, *what*

In your repo that's mainly:

- `docs/dev-specs/prompts/spec-new.md` ? new spec? instructions (rubric-aligned).
 - 2. **The orchestration template** ? how those rules are combined with PR data before the model runs.
- In your repo that's:

- `.github/prompts/dev-spec-generate.prompt.yml` ? stitches in PR body, slug, issue links, owners,

- Plus the **system** message in that same file (high-level ?obey instructions, exact headings?).

Together, those are what you'd paste into Gradescope (or link as GitHub file URLs) under **prompts**

?Make use of the prompts you developed in P2?
P2 had you use an LLM for **user stories, UI, and first implementation**. The course wants you to

- Reuse phrases / structure from whatever prompt doc or chat you used in P2 (story template, AC
 - Fold that into `spec-new.md` so the **dev spec** reads like a continuation of how you prompted P2.
- ### What you do **not** need to submit as ?the prompt?
- The **GitHub Actions YAML** alone is not the prompt; it?s automation. Graders care about the **LLM**

If you want a one-sentence answer for your write-up:
?Our new-spec prompt is versioned in `docs/dev-specs/prompts/spec-new.md`, composed with PR

[REDACTED]

--- Turn 75 (user) ---

The prompts used to update an existing development specification after the user story or the code

i also need prompt for this so...

--- Turn 76 (assistant) ---

Reviewing `spec-update.md` and strengthening it so it's a clear, submission-ready "update" prompt

[REDACTED]

--- Turn 77 (assistant) ---

Expanding `spec-update.md` into a self-contained, Gradescope-ready ?update? prompt (P2-aligned) and

[REDACTED]

--- Turn 78 (assistant) ---

[REDACTED]

--- Turn 79 (assistant) ---

Here?s what that assignment line means and what you have now.

What they want for ?update?

A **separate prompt document** that tells the LLM how to **revise an existing** dev spec when the

What you have in the repo (turn-in material)

Role	File
	----- -----
Update prompt	`docs/dev-specs/prompts/spec-update.md` ? expanded so it?s self-contained: P2
New prompt	`docs/dev-specs/prompts/spec-new.md` (already there)
Which is which for Gradescope	`docs/dev-specs/prompts/README.md` ? short table: new vs
How it?s wired in CI	The workflow still picks update vs new by whether `docs/dev-

Commit

Pushed on branch `feature/dev-spec-automation`: **d5bf0f3** (`docs: expand update dev-spec prompt

For Gradescope you can submit **exports** or **GitHub URLs** of:

- ``docs/dev-specs/prompts/spec-new.md`` ? ?prompts used to **generate** a new??
- ``docs/dev-specs/prompts/spec-update.md`` ? ?prompts used to **update**??
- Optionally ``github/prompts/dev-spec-generate.prompt.yml`` as ?how we compose PR context + prompts

[REDACTED]

--- Turn 80 (user) ---

this is what was reviewed on github:

Summary

This PR adds an approval-triggered workflow that uses GitHub Models to generate/update docs/dev-

Issues

Severity	File	Finding
BLOCKER	<code>github/workflows/dev-spec-on-pr-approval.yml</code> :L91-L95	Fetching HEAD_SHA from origin
WARN	<code>github/workflows/dev-spec-on-pr-approval.yml</code> :L41-L44,L77,L139	dev-spec slug unsanitized;
WARN	<code>github/workflows/dev-spec-on-pr-approval.yml</code> :L148-L156	Re-approvals can create

Action items

Fetch PR head from `github.event.pull_request.head.repo.full_name` (or use checked-out merge ref) for

Sanitize slug to a strict allowlist (e.g., `^[A-Za-z0-9._-]+$`) before file/branch usage.

Before `gh pr create`, detect existing open docs PR for same slug/PR and update/reuse it.

--- Turn 81 (assistant) ---

Implementing review fixes: fetch PR head via ``pull/N/head`` for fork compatibility, sanitize slugs to

[REDACTED]

--- Turn 82 (assistant) ---

Implementing all three review items: fetching PR head via ``refs/pull/N/head`` (fork-safe), strict

[REDACTED]

--- Turn 83 (assistant) ---

Normalizing slug case (e.g. ``us5`` ? ``US5``) and updating the README.

[REDACTED]

--- Turn 84 (assistant) ---

[REDACTED]

--- Turn 85 (assistant) ---

Here?s how the **GitHub review** items were addressed (committed as ``089b179`` on ``feature/dev-`

```
### 1. BLOCKER ? fork PRs: `git fetch origin $HEAD_SHA` fails
Change: Stop fetching `HEAD_SHA` from `origin`. After `git fetch origin "$BASE_SHA"`, the

```bash
git fetch origin "+refs/pull/${PR_NUM}/head:refs/remotes/origin/pr-${PR_NUM}-head"
```
```

and diffs against ``refs/remotes/origin/pr-${PR_NUM}-head``, which works for **fork PRs** because

2. WARN ? unsanitized slug / bad paths

Change: After resolving the slug from text or `dev-spec:` label, it must match

3. WARN ? re-approvals duplicate docs PRs

Change:

- Docs branch is now **stable:** `docs/dev-spec-\${SLUG}-pr-\${PR\_NUM}` (no `run\_id` suffix).
- After commit: **first push** uses `git push -u`; later pushes use **--force-with-lease** so the
- **gh pr create** runs only when `gh pr list --head "\$BRANCH" --state open` returns **zero** open

Docs

`docs/dev-specs/README.md` now mentions **fork-safe diffs** and **stable branch / re-approval**

Note: `git reset --hard` + `git clean -fd` at the start of the commit step clears untracked prep

You can reply on the GitHub review thread with: *Addressed in 089b179 ? pull/N/head fetch, slug*

[REDACTED]

--- **Turn 86 (user)** ---

The prompts used to generate a new development specification. Make use of the prompts you developed

The prompts used to update an existing development specification after the user story or the code

what do i submit for this

--- **Turn 87 (assistant)** ---

For those two bullets, graders want **the actual LLM instruction text** you use for **(1) first-**

What to submit (from this repo)

(1) ?Prompts used to generate a new development specification?

Submit the contents (or stable GitHub URLs) of:

- `docs/dev-specs/prompts/spec-new.md`
- Plus the **composed template** that wraps PR context + that file (so they see how it's fed to the
- `.github/prompts/dev-spec-generate.prompt.yml` (especially the `system` + `user` message structure

(2) ?Prompts used to update an existing development specification??

Submit:

- `docs/dev-specs/prompts/spec-update.md`
- Same composed template file above (it's shared for new vs update; the workflow swaps which

Optional but helpful index (makes grading obvious):

- `docs/dev-specs/prompts/README.md` ? explicitly labels which file is ?new? vs ?update?.

?Make use of the prompts you developed in P2?

Gradescope usually wants **evidence you reused P2 prompting style**, not that the files are

- A short **cover note** (1 paragraph) explaining how `spec-new.md` / `spec-update.md` reuses your

- A **short excerpt** from your P2 PDF / P2 chat log + a sentence mapping which phrases you carried

Practical packaging (pick one)

- **Best:** PDF with three parts: `spec-new.md`, `spec-update.md`, `dev-spec-generate.prompt.yml`

- **Also fine:** Public **GitHub blob links** to those files on `main` **plus** a 1-page PDF cover

You do **not** need to submit the whole workflow YAML as the `prompt`, but you **can** attach it as

[REDACTED]

--- Turn 88 (user) ---

ok comit those changes

--- Turn 89 (assistant) ---

[REDACTED]

--- Turn 90 (assistant) ---

[REDACTED]

--- Turn 91 (assistant) ---

There's **nothing** left to commit ? the working tree is clean on **feature/dev-spec-automation**.

Recent commits already include the work you care about, through **089b179** (fork-safe fetch, slug

If you meant **push** to GitHub so your open PR updates:

```
```bash
```

```
git push origin feature/dev-spec-automation
```

```
```
```

I can run that push from here if you want.

[REDACTED]

--- Turn 92 (user) ---

does the current yml file include committing the dev specs into code base

--- Turn 93 (assistant) ---

Partly yes, partly no ? it depends what you mean by **into the codebase**?

What the workflow **does** do

After the LLM output passes validation, the job:

1. Checks out `main` (as `origin/main`).
2. Creates/updates a branch named like `docs/dev-spec-US4-pr-123`.
3. Writes `docs/dev-specs/US4-development-spec.md` (or whatever slug).
4. **git add** + **git commit** that file.
5. **git push** to `origin` on that branch.
6. Runs **gh pr create** to open a **pull request** into `main` ? **only if** there isn't already

So the generated dev spec **is committed to Git** on a **docs branch** and pushed to GitHub.

What it **does not** do by itself

It does **not** merge straight into `main` with no review. The spec lands on **main** only after

Summary: yes ? it **commits and pushes** the dev spec into the **repository**; no ? it does

P2 story/

into these

(trim if hu

sheet list

supportin

sanitizati

an open

someone

not si

[REDACTED]

--- Turn 94 (user) ---

also i need ot do this:

automation should also create a issue to track creation of dev specs

--- Turn 95 (assistant) ---

Adding workflow steps to create a tracking issue (with a stable label and deduplication) and

[REDACTED]

--- Turn 96 (assistant) ---

[REDACTED]

--- Turn 97 (assistant) ---

Adding `issues: write`, creating a `dev-spec-tracking` label if missing, opening one tracking issue

[REDACTED]

--- Turn 98 (assistant) ---

[REDACTED]

--- Turn 99 (assistant) ---

[REDACTED]

--- Turn 100 (assistant) ---

Here's what was added.

Workflow (`.github/workflows/dev-spec-on-pr-approval.yml`)

- `permissions: issues: write` so the job can create issues.
- After a real spec change is staged (and `before` `git commit`):
- Ensures label `dev-spec-tracking` exists (`gh label create` if needed).
- Looks for an `open` issue with that label and title
- `Dev spec tracking: USn for feature PR #<feature-pr>`.
- If none ? `gh issue create` with a short body (feature PR, spec path, docs branch, workflow run).
- If one exists ? reuses its number.
- `git commit` includes `Refs #<issue>` so the commit links to the tracking issue.
- `New docs PR:` body includes `Tracking issue: #<issue>`; a `comment` is added on the issue
- `Existing docs PR (re-approval):` a `comment` is added on the same issue that the branch /

Docs (`.docs/dev-specs/README.md`)

- Short `Tracking issue?` section describing the behavior.

Commit

`c20c4a8` on `feature/dev-spec-automation` ? push when ready.

Note: The first run on a repo will create the `dev-spec-tracking` label automatically (needs

[REDACTED]

--- Turn 101 (user) ---

(base) evelynlo@MacBook-Air-212 GrocerEase % git push origin feature/dev-spec-automation

Enumerating objects: 15, done.

Counting objects: 100% (15/15), done.

Delta compression using up to 10 threads

Compressing objects: 100% (8/8), done.

Writing objects: 100% (8/8), 1.75 KiB | 1.75 MiB/s, done.
Total 8 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), completed with 6 local objects.
To https://github.com/gsha22/GrocerEase.git
089b179..c20c4a8 feature/dev-spec-automation -> feature/dev-spec-automation

why is there no pr remade

--- Turn 102 (assistant) ---

`git push` **only** updates the branch on GitHub. It does **not** create or remake a pull request

What usually happened in your case

1. **If a PR from `feature/dev-spec-automation` to `main` is still open**

There is **no new PR** the **same** PR now includes commits `089b179..c20c4a8`. Open that PR in

2. **If that PR was merged or closed**

Pushing the branch again **does not** reopen it. You open a **new** PR:

- GitHub: branch page ? **Compare & pull request?**, or

- Terminal:

`gh pr create --base main --head feature/dev-spec-automation`

3. **The dev-spec automation PR**

Your workflow **does not** run when you push `feature/dev-spec-automation`. It runs on

Quick check

```
```bash
```

```
gh pr list --head feature/dev-spec-automation --state all
```

```
```
```

If you see an **open** PR, that's the one that was updated. If you see **none** open, create a new

[REDACTED]

--- Turn 103 (user) ---

can u check if theres any difference between main and my current local code base

--- Turn 104 (assistant) ---

[REDACTED]

--- Turn 105 (assistant) ---

[REDACTED]

--- Turn 106 (assistant) ---

[REDACTED]

--- Turn 107 (assistant) ---

Here's what your repo looks like **right now** (`feature/dev-spec-automation` vs `origin/main``):

Branch / commits

- **Current branch:** `feature/dev-spec-automation``

- **You are behind `main` by 5 commits** (these are on `main`` but not ancestors of your current

- `1f02c14` Merge PR ##133` (feature/dev-spec-automation` to `main`)`

- `b2e383a` Merge PR ##127` (docs/p4-code-gen-automation`)`

- `b0875d7`, `ad23b3f`, `5c22a86` (P4 docs / procedure commits)`

- **You are ahead of `main` by 1 commit** (only on your branch):

- `c20c4a8` ? *ci: create dev-spec tracking issue and link docs PR*

So histories **diverged** at merge-base **089b179**: `main`` moved forward (including merging #133

and other

What that means in plain language

- `main`` already contains the earlier dev-spec automation work via **PR #133**.
- Your **latest** change (**tracking issue**, `c20c4a8``) is **not on `main` yet** until you

open/me

How to sync (typical)

```
``bash
git fetch origin
git merge origin/main
# resolve conflicts if any, then
git push origin feature/dev-spec-automation
``
```

(or `git rebase origin/main`` if your team prefers linear history)

If you want, I can run `git merge origin/main`` for you in the repo and report conflicts.

[REDACTED]

--- Turn 108 (user) ---

<attached_files>

<code_selection path="/Users-evelynlo-Desktop-GrocerEase/1" lines="1-18">

```
Merge remote-tracking branch 'origin/main' into feature/dev-spec-automation
# Please enter a commit message to explain why this merge is necessary,
# especially if it merges an updated upstream into a topic branch.
#
# Lines starting with '#' will be ignored, and an empty message aborts
# the commit.
```

~
~
~
10|~
~
~
~
~
~
~
~
~

"~/Desktop/GrocerEase/.git/MERGE_MSG" 6L, 302B

</code_selection>

<terminal_selection title="Terminal" path="/Users/evelynlo/.cursor/projects/Users-evelynlo-Desktop-

GrocerEa

```
Merge remote-tracking branch 'origin/main' into feature/dev-spec-automation
# Please enter a commit message to explain why this merge is necessary,
# especially if it merges an updated upstream into a topic branch.
#
# Lines starting with '#' will be ignored, and an empty message aborts
# the commit.
```

~
~
~

~
~
~
~
~
~
~
~

"~/Desktop/GrocerEase/.git/MERGE_MSG" 6L, 302B

</terminal_selection>

</attached_files>

<user_query>

@/Users/evelynlo/.cursor/projects/Users-evelynlo-Desktop-GrocerEase/terminals/1.txt:7-24

whattttt i should just open a new pr for the new commit right

</user_query>

--- Turn 109 (assistant) ---

You're not choosing between **merge** and **new PR** ? they do different jobs:

What that screen is

Git opened an editor for the **merge commit message** because you ran something like `git merge`

In Vim: save and exit with `:wq` (default first line is fine as the merge message).

To **cancel** the merge instead: `q!` then in shell run `git merge --abort`.

Do you just open a new PR??

`main` already merged PR #133 from this branch, so that old PR is **closed**. Your new work

1. Finish the merge (or rebase onto `main`), then
2. `git push origin feature/dev-spec-automation`, then
3. Open a **new** PR to `main` (GitHub UI or `gh pr create`).

So: **finish merge** ? **push** ? **new PR** is the normal path. You don't skip the merge unless you prefer

[REDACTED]

--- Turn 110 (user) ---

wait i dont want to merge to main directly, i want to create a new pr for it

--- Turn 111 (assistant) ---

You're mixing up two different things:

`git merge origin/main` (what you started)

That only merges `main` into your current branch (`feature/dev-spec-automation`) **on your**

It does **not** put your work on `main`. It does **not** replace opening a PR. It just brings your

Opening a new PR?

That's how you ask to merge **your branch** ? `main` **through GitHub** (review, checks, merge

Order: finish the merge into your feature branch ? `git push` that branch ? **open a new PR**

So: you are **not** merging to `main` directly by running `git merge origin/main`. You're preparing

If you'd rather not have a merge commit on your branch, you can `git merge --abort` and instead

[REDACTED]

--- Turn 112 (user) ---

<attached_files>

<code_selection path="/Users-evelynlo-Desktop-GrocerEase/1" lines="61-68">

(base) evelynlo@MacBook-Air-212 GrocerEase % git fetch origin

(base) evelynlo@MacBook-Air-212 GrocerEase % git merge origin/main

Merge made by the 'ort' strategy.

docs/p4-code-generation-procedure.md | 211

docs/p4-testing-procedure.md | 209

2 files changed, 420 insertions(+)

create mode 100644 docs/p4-code-generation-procedure.md

create mode 100644 docs/p4-testing-procedure

</code_selection>

<terminal_selection title="Terminal" path="/Users/evelynlo/.cursor/projects/Users-evelynlo-Desktop-

(base) evelynlo@MacBook-Air-212 GrocerEase % git fetch origin

(base) evelynlo@MacBook-Air-212 GrocerEase % git merge origin/main

Merge made by the 'ort' strategy.

docs/p4-code-generation-procedure.md | 211

docs/p4-testing-procedure.md | 209

2 files changed, 420 insertions(+)

create mode 100644 docs/p4-code-generation-procedure.md

create mode 100644 docs/p4-testing-procedure

</terminal_selection>

</attached_files>

<user_query>

@/Users/evelynlo/.cursor/projects/Users-evelynlo-Desktop-GrocerEase/terminals/1.txt:67-74

ok what do i do now to create the new pr for the new commit

</user_query>

--- Turn 113 (assistant) ---

Your merge is done locally. Next you only need to **push the branch**, then **open a PR** into

`main` (th

1. Push your branch

```bash

git push origin feature/dev-spec-automation

```

2. Create the new PR (pick one)

****Option A ? GitHub CLI****

```bash

gh pr create --base main --head feature/dev-spec-automation \

--title "ci: dev-spec tracking issue + sync with main" \

--body "Follow-up after #133: adds dev-spec tracking issue automation and merges latest main.

- Tracking issue creation + `Refs` in spec commits

- (describe anything else you want reviewers to see)"

---

**\*\*Option B ? Browser\*\***

After the push, open the repo on GitHub ? you should see a yellow **\*\*?Compare & pull request?\*\***

---

**\*\*Note:\*\*** The merge you ran only added the two P4 doc files from `main` onto your branch; your

If `git push` errors (e.g. rejected), paste the message and we can fix (sometimes need `--force-

[REDACTED]

**--- Turn 114 (user) ---**

Summary

This PR adds automated tracking issue creation/linking to the dev-spec workflow and updates docs

Issues

Severity    File    Finding

WARN    .github/workflows/dev-spec-on-pr-approval.yml (around L273)    gh issue list uses default

WARN    .github/workflows/dev-spec-on-pr-approval.yml (around L323)    Existing PR lookup assumes

Action items

Add pagination (--limit/--paginate) when searching existing tracking issues to avoid duplicates.

Validate DOCS\_PR is non-empty before issue comment; fail fast with clear error.

**--- Turn 115 (user) ---**

<agent\_transcripts>

Agent transcripts (past chats) live in /Users/evelynlo/.cursor/projects/Users-evelynlo-Desktop-

</agent\_transcripts>

<agent\_skills>

When users ask you to perform tasks, check if any of the available skills below can help complete

<available\_skills description="Skills the agent can use. Use the Read tool with the provided

<agent\_skill fullPath="/Users/evelynlo/.cursor/skills-cursor/babysit/SKILL.md">Keep a PR merge-ready

<agent\_skill fullPath="/Users/evelynlo/.cursor/skills-cursor/canvas/SKILL.md" />

<agent\_skill fullPath="/Users/evelynlo/.cursor/skills-cursor/create-hook/SKILL.md">Create Cursor

<agent\_skill fullPath="/Users/evelynlo/.cursor/skills-cursor/create-rule/SKILL.md">Create Cursor

<agent\_skill fullPath="/Users/evelynlo/.cursor/skills-cursor/create-skill/SKILL.md">Guides users

<agent\_skill fullPath="/Users/evelynlo/.cursor/skills-cursor/statusline/SKILL.md">Configure a custom

status lin

<agent\_skill fullPath="/Users/evelynlo/.cursor/skills-cursor/update-cli-config/SKILL.md">View and

modify C

<agent\_skill fullPath="/Users/evelynlo/.cursor/skills-cursor/update-cursor-settings/SKILL.md">Modify

Cursor/V

</available\_skills>

</agent\_skills>

--- Turn 116 (user) ---

Summary

This PR adds automated tracking issue creation/linking to the dev-spec workflow and updates docs

according

Issues

Severity File Finding

WARN .github/workflows/dev-spec-on-pr-approval.yml (around L273) gh issue list uses default

page size

WARN .github/workflows/dev-spec-on-pr-approval.yml (around L323) Existing PR lookup assumes

first resu

Action items

Add pagination (--limit/--paginate) when searching existing tracking issues to avoid duplicates.

Validate DOCS\_PR is non-empty before issue comment; fail fast with clear error.

--- Turn 117 (assistant) ---

Fixing both review items: increasing `gh issue list` limit (and using safe jq parsing) and

validating

[REDACTED]

--- Turn 118 (assistant) ---

[REDACTED]

--- Turn 119 (assistant) ---

[REDACTED]

--- Turn 120 (assistant) ---

[REDACTED]

--- Turn 121 (assistant) ---

Replacing `--paginate` on `gh issue list` ? the CLI manual lists `--limit` but not `--paginate` for

that com

[REDACTED]

--- Turn 122 (assistant) ---

[REDACTED]

--- Turn 123 (assistant) ---

[REDACTED]

--- Turn 124 (assistant) ---

[REDACTED]

--- Turn 125 (assistant) ---



Here is what was changed to address the review.

### ### 1. Tracking issue lookup (pagination / limit)

`gh issue list` defaults to **30** items. The workflow now uses **--limit 500** so a matching

```
273:275:/Users/evelynlo/Desktop/GrocerEase/.github/workflows/dev-spec-on-pr-approval.yml
Raise limit so an older matching issue beyond the default page (30) is not missed (duplicate
```

```
EXISTING_ISSUE=$(gh issue list --repo "$GITHUB_REPO" --state open --label "$TRACK_LABEL" --limit 500
```

```
| jq -r --arg t "$ISSUE_TITLE" '[] | select(.title == $t) | first | .number // empty')
...
```

`gh issue list` does not document a **--paginate** flag the way `gh api` does; **--limit 500**

### ### 2. `DOCS\_PR` validation (fail fast)

- **gh pr list** for the open-count check now uses **--limit 50** so the count is not capped at

- After **gh pr create**, the run **exits** with **::error::** if the PR number is empty.

- On the **?existing PR?** path, **jq** uses **.[0].number // empty**; if **DOCS\_PR** is still

```
309:341:/Users/evelynlo/Desktop/GrocerEase/.github/workflows/dev-spec-on-pr-approval.yml
```

```
OPEN_COUNT=$(gh pr list --repo "$GITHUB_REPO" --head "$BRANCH" --state open --limit 50 --json number
```

```
if ["$OPEN_COUNT" -eq 0]; then
```

```
DOCS_PR=$(gh pr create --repo "$GITHUB_REPO" --base main --head "$BRANCH" \
```

```
...
```

```
--json number -q .number)
```

```
if [-z "${DOCS_PR:-}"]; then
```

```
echo "::error::gh pr create succeeded but returned no PR number (check gh version / API)."
```

```
exit 1
```

```
fi
```

```
gh issue comment "$ISSUE_NUM" --repo "$GITHUB_REPO" \
```

```
...
```

```
else
```

```
DOCS_PR=$(gh pr list --repo "$GITHUB_REPO" --head "$BRANCH" --state open --limit 20 --json number
```

```
if [-z "${DOCS_PR:-}"]; then
```

```
echo "::error::Expected an open docs PR for head \"${BRANCH}\" (OPEN_COUNT was ${OPEN_COUNT}) but gh
```

```
exit 1
```

```
fi
```

```
gh issue comment "$ISSUE_NUM" --repo "$GITHUB_REPO" \
```

```
...
```

```
fi
```

```
...
```

You can commit and push this on your dev-spec automation branch as a small follow-up to the same PR.

[REDACTED]

--- Turn 126 (user) ---

<attached\_files>

<code\_selection path="/Users-evelynlo-Desktop-GrocerEase/1" lines="80-84">

```
80|(base) evelynlo@MacBook-Air-212 GrocerEase % git add .
(base) evelynlo@MacBook-Air-212 GrocerEase % git commit -m "fix paginate issues"
[feature/dev-spec-automation 6446fbf] fix paginate issues
1 file changed, 13 insertions(+), 4 deletions(-)
(base) evelynlo@MacBook-Air-212 GrocerEase %
</code_selection>
```

```
<terminal_selection title="Terminal" path="/Users/evelynlo/.cursor/projects/Users-evelynlo-Desktop-
```

```
(base) evelynlo@MacBook-Air-212 GrocerEase % git add .
(base) evelynlo@MacBook-Air-212 GrocerEase % git commit -m "fix paginate issues"
[feature/dev-spec-automation 6446fbf] fix paginate issues
1 file changed, 13 insertions(+), 4 deletions(-)
(base) evelynlo@MacBook-Air-212 GrocerEase %
</terminal_selection>
```

```
</attached_files>
```

```
<user_query>
```

```
@/Users/evelynlo/.cursor/projects/Users-evelynlo-Desktop-GrocerEase/terminals/1.txt:86-90
```

how to push this to the pr

```
</user_query>
```

### --- Turn 127 (assistant) ---

Pushing the branch updates the open PR automatically if the PR's head branch is `feature/dev-spec-

[REDACTED]

### --- Turn 128 (assistant) ---

[REDACTED]

### --- Turn 129 (assistant) ---

**\*\*Pushed.\*\*** Your commit `6446fbf` is on `origin/feature/dev-spec-automation`, so any PR that uses

For next time, from this repo:

```
```bash
```

```
git push -u origin feature/dev-spec-automation
```

```
```
```

`-u` sets upstream tracking (already done now). After that, `git push` alone is enough.

If the PR is from a **\*\*fork\*\*** under your user, push to **\*\*your\*\*** fork's remote instead of `origin`,

[REDACTED]

### --- Turn 130 (user) ---

ok create a new branch for manually generate the dev specs for us15:

here is the us15 issue in full:

US 15: Shopper-Facing Store Review or Rating #119

Closed

#126

Closed

US 15: Shopper-Facing Store Review or Rating

#119

#126

@AvalonMei

Description

AvalonMei

opened 3 days ago

As a shopper, I want to leave a short rating or note on a store I've visited so that other shoppers

can make

Why This Is Important

Discovery interviews revealed that shoppers often hesitate to visit local stores because they lack

social pro

Human Acceptance Criteria

A logged-in shopper can leave a rating in under 30 seconds from the store profile page

The average rating and note count are visible to all visitors including guests

A shopper who has already rated a store sees their existing rating with an option to delete it, not

a second

Notes feel lightweight ? more like a quick tip than a formal review

Machine Acceptance Criteria

POST /api/stores/:id/ratings with a score (1?5) and optional note (max 280 characters) returns 201;

missing c

A shopper can only submit one rating per store; a duplicate returns 409

GET /api/stores/:id/ratings returns the average score, total count, and most recent notes

(paginate

DELETE /api/stores/:id/ratings/:ratingId allows a shopper to remove their own rating; attempting to

delete an

Unauthenticated requests to POST or DELETE return 401; GET is public

we already merged the pr for this user story, so i just want you to generate a new branch for

creating

US15: Shopper-facing store ratings (#119)

#126

Merged

AvalonMei

merged 4 commits into

main

from

feature/us-15-store-ratings

yesterday

+805

Lines changed: 805 additions & 0 deletions

Conversation7 (7)

Commits4 (4)

Checks5 (5)

Files changed8 (8)

Merged

US15: Shopper-facing store ratings (#119)#126

AvalonMei

merged 4 commits into

main

from

feature/us-15-store-ratings

Conversation

@gsha22

gsha22

commented

yesterday  
Closes #119

## Summary

Lightweight 1?5 rating + optional 280-char note per shopper per store.

Aggregates are public; write/delete require a shopper session.

Prisma model `StoreRating` with `@@unique([storeId, shopperId])` so a shopper holds at most one

`GET /api/stores/[id]/ratings` ? public, returns `{average, total, hasMore, ratings:[?]}`; 10 per

`POST /api/stores/[id]/ratings` ? shopper-only; 400 on missing/out-of-range score; 409 on duplicate

`DELETE /api/stores/[id]/ratings/[ratingId]` ? shopper-only; 403 if the rating belongs to another

`StoreRatingsPanel` island on `/stores/[id]` ? star picker, optional note, "Your rating" card with

## Machine acceptance criteria

1?5 score required, optional 280-char note.

Duplicate ? 409; unauth POST/DELETE ? 401; GET public.

Average + total + paginated notes (10/page) returned.

DELETE of someone else's rating ? 403.

## Human acceptance criteria

Rate in under 30 s from the store page (inline panel, 5 stars, optional note).

Public visitors see aggregate.

Already-rated shoppers see their own rating with Delete (no second submit form).

## Automation note

Implemented via Claude Code (Opus 4.7) as part of P4 task 3. Prompt/procedure documented in the P4

? Generated with Claude Code

@gsha22

US15: Shopper-facing store ratings (closes #119)

d81909e

@github-actions

github-actions bot

commented

yesterday

## Summary

This PR adds end-to-end store ratings: schema, public aggregate/read API, shopper-only create/delete

## Issues

Severity File Finding

WARN prisma/migrations/20260419130000\_store\_ratings/migration.sql:L6 No DB CHECK constraint

INFO app/api/stores/[id]/ratings/route.ts:L33-L44 ?Recent notes? query returns ratings without

## Action items

Add a database constraint enforcing score BETWEEN 1 AND 5 in migration/schema.

Filter ratings feed to note-bearing entries (e.g., where: { note: { not: null } }).

Automated review: GitHub Copilot CLI in GitHub Actions.

\_Workflow: Copilot PR review ? Run: <https://github.com/gsha22/GrocerEase/actions/runs/24637074868>

@gsha22 gsha22 mentioned this pull request yesterday

docs: P4 code-generation procedure #127

Merged

@gsha22  
test(us15): Jest coverage for ratings panel  
a981440  
@github-actions  
github-actions bot  
commented  
yesterday  
Summary  
This PR adds end-to-end shopper store ratings: schema, public aggregate/read API, shopper-gated

create/de

Issues  
Severity   File   Finding  
WARN   components/StoreRatingsPanel.tsx (L95-L111)   refresh() swallows fetch errors, leaving  
WARN   app/api/stores/[id]/ratings/route.ts (L33-L47)   ?Recent notes? endpoint paginates all  
Action items  
Surface refresh failures in StoreRatingsPanel (or rollback optimistic UI) instead of silent catch.  
If requirement is truly ?recent notes,? filter findMany by non-null/non-empty notes and align

stale sun

ratings, i

total/has

Automated review: GitHub Copilot CLI in GitHub Actions.  
\_Workflow: Copilot PR review ? Run: <https://github.com/gsha22/GrocerEase/actions/runs/24637932006>

@AvalonMei  
AvalonMei  
commented  
yesterday  
Can you fix the issues brought up by Copilot?

@gsha22  
fix(us15): DB score constraint, notes-only pagination, surface refres?  
01650b5  
@github-actions  
github-actions bot  
commented  
yesterday  
Summary  
This PR adds end-to-end shopper store ratings: schema/migrations, public aggregate/read APIs,

shopper-

Issues  
Severity   File   Finding  
WARN   app/(public)/stores/[id]/page.tsx (around L88-L125)   SSR loads all ratings, not ?notes-  
WARN   app/(public)/stores/[id]/page.tsx (around L120-L125)   hasMore uses total ratings count,  
Action items  
Align SSR preload query with API behavior: filter to non-empty notes and paginate first 10 notes.  
Compute SSR hasMore from notes count (not total ratings) to avoid misleading UI state.  
Automated review: GitHub Copilot CLI in GitHub Actions.  
\_Workflow: Copilot PR review ? Run: <https://github.com/gsha22/GrocerEase/actions/runs/24638484085>

only? like

not notes

@AvalonMei  
AvalonMei  
commented

yesterday

Can you fix the merge conflicts and address the new issues copilot brought up?

@gsha22

merge origin/main into feature/us-15-store-ratings

b601259

@github-actions

github-actions bot

commented

yesterday

Summary

This PR cleanly adds shopper store ratings end-to-end: schema/migrations, public aggregate+notes

retrieval,

Issues

No issues found.

Action items

None blocking.

Automated review: GitHub Copilot CLI in GitHub Actions.

\_Workflow: Copilot PR review ? Run: <https://github.com/gsha22/GrocerEase/actions/runs/24643010180>

@AvalonMei

AvalonMei

commented

yesterday

LGTM

here is the file changed:

Skip to content

gsha22

GrocerEase

Repository navigation

Code

Issues

7

(7)

Pull requests

2

(2)

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

US15: Shopper-facing store ratings (#119)

#126

Merged

AvalonMei

merged 4 commits into

main

from

feature/us-15-store-ratings

yesterday

+805

Lines changed: 805 additions & 0 deletions

Conversation7 (7)

Commits4 (4)

Checks5 (5)

Files changed8 (8)

Pull Request Toolbar

Merged

US15: Shopper-facing store ratings (#119)

#126

AvalonMei

merged 4 commits into

main

from

feature/us-15-store-ratings

0 / 8 viewed

Filter files?

File tree

app

(public)/stores/[id]

page.tsx

api/stores/[id]/ratings

[ratingId]

route.ts

route.ts

components

StoreRatingsPanel.tsx

prisma

migrations

20260419130000\_store\_ratings

migration.sql

20260419140000\_store\_ratings\_score\_check

migration.sql

schema.prisma

tests

StoreRatingsPanel.test.tsx

?app/(public)/stores/[id]/page.tsx?

+71

Lines changed: 71 additions & 0 deletions

Original file line number   Diff line number   Diff line change

import ItemAvailabilitySearch from "@components/ItemAvailabilitySearch";

import StoreAlertSubscribe from "@components/StoreAlertSubscribe";

import StoreFreshUpdatesFeed from "@components/StoreFreshUpdatesFeed";

import StoreRatingsPanel from "@components/StoreRatingsPanel";

import StoreReportButton from "@components/StoreReportButton";

export const dynamic = "force-dynamic";

storeFollowAlertId = follow?.id ?? null;

}

const RATINGS\_PAGE\_SIZE = 10;

// Mirror GET /api/stores/[id]/ratings: `total` counts all ratings (for the

// "X ratings" header), but the preloaded feed and `hasMore` only consider

// ratings that actually carry a note ? otherwise the UI would render

// empty rows and paginate past real content.

```

const notesWhere = {
 storeId: id,
 note: { not: null },
 NOT: { note: "" },
} as const;

const [ratingAgg, recentNotesCount, recentNotes] = await Promise.all([
 prisma.storeRating.aggregate({
 where: { storeId: id },
 _avg: { score: true },
 _count: { _all: true },
 }),
 prisma.storeRating.count({ where: notesWhere }),
 prisma.storeRating.findMany({
 where: notesWhere,
 orderBy: { createdAt: "desc" },
 take: RATINGS_PAGE_SIZE,
 select: {
 id: true,
 score: true,
 note: true,
 createdAt: true,
 shopper: { select: { name: true } },
 },
 }),
]);

const ratingsSummary = {
 average:
 ratingAgg._avg.score != null
 ? Math.round(ratingAgg._avg.score * 10) / 10
 : null,
 total: ratingAgg._count._all,
 ratings: recentNotes.map((r) => ({
 id: r.id,
 score: r.score,
 note: r.note,
 createdAt: r.createdAt.toISOString(),
 authorName: r.shopper.name,
 })),
 hasMore: recentNotesCount > RATINGS_PAGE_SIZE,
};

let initialOwnRating: {
 id: string;
 score: number;
 note: string | null;
} | null = null;

if (session?.role === "shopper") {
 const own = await prisma.storeRating.findUnique({
 where: {
 storeId_shopperId: { storeId: id, shopperId: session.user.id },
 },
 select: { id: true, score: true, note: true },
 });

 if (own) initialOwnRating = own;
}

```



```

const freshUpdatesDisplay = enrichFreshUpdatesWithStale(
store.freshUpdates,
asOf,
viewerRole={viewerRole}
/>

{/* Shopper ratings ? Story 15 */}
<StoreRatingsPanel
storeId={store.id}
viewerRole={viewerRole}
initialSummary={ratingsSummary}
initialOwnRating={initialOwnRating}
/>

{/* Fresh Today ? Story 1 */}
<div className="bg-white rounded-2xl border border-gray-200 p-6 mb-4">
<h2 className="text-[17px] font-semibold text-gray-800 mb-4 flex items-center gap-2">
?app/api/stores/[id]/ratings/[ratingId]/route.ts?
+31

```

Lines changed: 31 additions & 0 deletions

Original file line number    Diff line number    Diff line change

```

// Story 15: A shopper may delete their own rating only.
import { NextRequest, NextResponse } from "next/server";
import { prisma } from "@lib/prisma";
import { requireShopperSession } from "@lib/require-shopper-session";
export async function DELETE(
 _req: NextRequest,
 ctx: { params: Promise<{ id: string; ratingId: string }> },
) {
 const gate = await requireShopperSession();
 if (!gate.ok) return gate.response;
 const { session } = gate;
 const { id: storeId, ratingId } = await ctx.params;
 const rating = await prisma.storeRating.findUnique({
 where: { id: ratingId },
 select: { id: true, shopperId: true, storeId: true },
 });
 if (!rating || rating.storeId !== storeId) {
 return NextResponse.json({ error: "Rating not found" }, { status: 404 });
 }
 if (rating.shopperId !== session.user.id) {
 return NextResponse.json(
 { error: "You can only delete your own rating." },
 { status: 403 },
);
 }
 await prisma.storeRating.delete({ where: { id: ratingId } });
 return NextResponse.json({ ok: true });
}
?app/api/stores/[id]/ratings/route.ts?
+155

```

Lines changed: 155 additions & 0 deletions

Original file line number    Diff line number    Diff line change

```

// Story 15: Shopper-facing store ratings.
// - GET is public; returns aggregate + paginated recent notes.

```

```

// - POST requires a shopper session; one rating per (store, shopper).
import { NextRequest, NextResponse } from "next/server";
import { Prisma } from "@app/generated/prisma/client";
import { prisma } from "@lib/prisma";
import { requireShopperSession } from "@lib/require-shopper-session";

const PAGE_SIZE = 10;
const NOTE_MAX = 280;
export async function GET(
 req: NextRequest,
 ctx: { params: Promise<{ id: string }> },
) {
 const { id: storeId } = await ctx.params;
 const page = Math.max(1, parseInt(req.nextUrl.searchParams.get("page") ?? "1", 10) || 1);
 const store = await prisma.store.findUnique({
 where: { id: storeId },
 select: { id: true },
 });
 if (!store) {
 return NextResponse.json({ error: "Store not found" }, { status: 404 });
 }
 const agg = await prisma.storeRating.aggregate({
 where: { storeId },
 _avg: { score: true },
 _count: { _all: true },
 });
 // The paginated feed is "recent notes" ? only ratings that carried a
 // written note. `total` still reflects all ratings (used for the "X
 // ratings" header count), but pagination/hasMore tracks notes only.
 const noteWhere = {
 storeId,
 note: { not: null },
 NOT: { note: "" },
 } as const;
 const [notesCount, ratings] = await Promise.all([
 prisma.storeRating.count({ where: noteWhere }),
 prisma.storeRating.findMany({
 where: noteWhere,
 orderBy: { createdAt: "desc" },
 skip: (page - 1) * PAGE_SIZE,
 take: PAGE_SIZE,
 select: {
 id: true,
 score: true,
 note: true,
 createdAt: true,
 shopper: { select: { name: true } },
 },
 }),
]);
 const total = agg._count._all;
 const average =
 agg._avg.score != null ? Math.round(agg._avg.score * 10) / 10 : null;
 return NextResponse.json({
 average,

```

```

total,
page,
pageSize: PAGE_SIZE,
hasMore: page * PAGE_SIZE < notesCount,
ratings: ratings.map((r) => ({
 id: r.id,
 score: r.score,
 note: r.note,
 createdAt: r.createdAt,
 authorName: r.shopper.name,
})),
});
}

export async function POST(
 req: NextRequest,
 ctx: { params: Promise<{ id: string }> },
) {
 const gate = await requireShopperSession();
 if (!gate.ok) return gate.response;
 const { session } = gate;
 const { id: storeId } = await ctx.params;
 const body = await req.json().catch(() => null);
 const score = body?.score;
 const noteRaw = body?.note;
 if (
 typeof score !== "number" ||
 !Number.isInteger(score) ||
 score < 1 ||
 score > 5
) {
 return NextResponse.json(
 { error: "score must be an integer between 1 and 5" },
 { status: 400 },
);
 }

 let note: string | null = null;
 if (noteRaw !== null) {
 if (typeof noteRaw !== "string") {
 return NextResponse.json(
 { error: "note must be a string" },
 { status: 400 },
);
 }

 const trimmed = noteRaw.trim();
 if (trimmed.length > NOTE_MAX) {
 return NextResponse.json(
 { error: `note too long (max ${NOTE_MAX} characters)` },
 { status: 400 },
);
 }

 note = trimmed || null;
 }

 const store = await prisma.store.findUnique({
 where: { id: storeId },
 });

```

```

select: { id: true },
});
if (!store) {
return NextResponse.json({ error: "Store not found" }, { status: 404 });
}
try {
const rating = await prisma.storeRating.create({
data: {
storeId,
shopperId: session.user.id,
score,
note,
},
select: { id: true, score: true, note: true, createdAt: true },
});
return NextResponse.json({ rating }, { status: 201 });
} catch (err) {
if (
err instanceof Prisma.PrismaClientKnownRequestError &&
err.code === "P2002"
) {
return NextResponse.json(
{
error:
"You've already rated this store. Delete your rating first to submit a new one.",
},
{ status: 409 },
);
}
throw err;
}
}

```

?components/StoreRatingsPanel.tsx?

+303

Lines changed: 303 additions & 0 deletions

Original file line number   Diff line number   Diff line change

// Story 15: Shopper-facing lightweight rating + tip panel on a store page.

"use client";

import Link from "next/link";

import { useEffect, useState } from "react";

import type { ViewerRole } from "@lib/viewer-role";

type RatingItem = {

id: string;

score: number;

note: string | null;

createdAt: string;

authorName: string;

};

type Summary = {

average: number | null;

total: number;

ratings: RatingItem[];

hasMore: boolean;

};

```

type OwnRating = {
 id: string;
 score: number;
 note: string | null;
} | null;

type Props = {
 storeId: string;
 viewerRole: ViewerRole;
 initialSummary: Summary;
 /** Own rating if the viewer is a shopper who has already rated. */
 initialOwnRating: OwnRating;
};

function Stars({ score, onPick }: { score: number; onPick?: (n: number) => void }) {
 return (
 <div className="flex items-center gap-0.5" role="group" aria-label="Rating stars">
 {[1, 2, 3, 4, 5].map((n) => {
 const filled = n <= score;
 const interactive = typeof onPick === "function";
 const className = `text-[20px] leading-none ${
 filled ? "text-amber-500" : "text-gray-300"
 } ${interactive ? "hover:scale-110 transition-transform" : ""}`;
 if (interactive) {
 return (
 <button
 key={n}
 type="button"
 aria-label={` ${n} star${n > 1 ? "s" : ""}`}
 aria-pressed={filled}
 onClick={() => onPick?.(n)}
 className={className}
 >
 ?
 </button>
);
 }
 return (

 ?

);
 });
]}
 </div>
);
}

function formatDate(iso: string) {
 return new Intl.DateTimeFormat("en-US", {
 month: "short",
 day: "numeric",
 year: "numeric",
 }).format(new Date(iso));
}

export default function StoreRatingsPanel({
 storeId,
 viewerRole,

```

```

initialSummary,
initialOwnRating,
}: Props) {
 const [summary, setSummary] = useState<Summary>(initialSummary);
 const [ownRating, setOwnRating] = useState<OwnRating>(initialOwnRating);
 const [score, setScore] = useState(0);
 const [note, setNote] = useState("");
 const [submitting, setSubmitting] = useState(false);
 const [error, setError] = useState<string | null>(null);

 useEffect(() => {
 setSummary(initialSummary);
 setOwnRating(initialOwnRating);
 }, [initialSummary, initialOwnRating, storeId]);

 async function refresh() {
 try {
 const res = await fetch(`/api/stores/${storeId}/ratings`, {
 cache: "no-store",
 });
 if (!res.ok) {
 throw new Error("Could not refresh ratings.");
 }
 const data = await res.json();
 setSummary({
 average: data.average,
 total: data.total,
 ratings: data.ratings,
 hasMore: data.hasMore,
 });
 } catch (e) {
 setError(
 e instanceof Error
 ? `${e.message} You may need to reload the page.`
 : "Could not refresh ratings. You may need to reload the page.",
);
 }
 }

 async function submit() {
 setError(null);
 if (score < 1 || score > 5) {
 setError("Pick 1-5 stars first.");
 return;
 }
 setSubmitting(true);
 try {
 const res = await fetch(`/api/stores/${storeId}/ratings`, {
 method: "POST",
 credentials: "include",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ score, note: note.trim() || undefined }),
 });
 const data = await res.json().catch(() => ({}));
 if (!res.ok) {
 throw new Error(data.error || "Could not submit rating.");
 }
 }
 }
}

```

```

setOwnRating({
id: data.rating.id,
score: data.rating.score,
note: data.rating.note,
});
setScore(0);
setNote("");
refresh();
} catch (e) {
setError(e instanceof Error ? e.message : "Could not submit rating.");
} finally {
setSubmitting(false);
}
}

async function deleteOwn() {
if (!ownRating) return;
setSubmitting(true);
setError(null);
try {
const res = await fetch(
`/api/stores/${storeId}/ratings/${ownRating.id}`,
{ method: "DELETE", credentials: "include" },
);
if (!res.ok) {
const data = await res.json().catch(() => ({}));
throw new Error(data.error || "Could not delete rating.");
}
setOwnRating(null);
refresh();
} catch (e) {
setError(e instanceof Error ? e.message : "Could not delete rating.");
} finally {
setSubmitting(false);
}
}

const callbackPath = `/stores/${storeId}`;
const shopperLoginHref = `/shopper/login?callbackUrl=${encodeURIComponent(callbackPath)}`;
return (
<section
className="bg-white rounded-2xl border border-gray-200 p-6 mb-4 space-y-4"
aria-labelledby="ratings-heading"
>
<div className="flex items-center justify-between gap-3 flex-wrap">
<h2
id="ratings-heading"
className="text-[17px] font-semibold text-gray-800 flex items-center gap-2"
>
?
Shopper ratings
</h2>
<div className="text-[13px] text-gray-500 flex items-center gap-2">
{summary.average != null ? (
<>
<Stars score={Math.round(summary.average)} />

```

```


{summary.average.toFixed(1)}

({summary.total} {summary.total === 1 ? "rating" : "ratings"})

</>
) : (
No ratings yet. Be the first!
)}
</div>
</div>
{viewerRole === "shopper" ? (
 ownRating ? (
 <div className="rounded-xl bg-emerald-50/60 border border-emerald-100 p-4">
 <p className="text-[12px] font-semibold text-emerald-900 uppercase tracking-wider">
 Your rating
 </p>
 <div className="mt-1 flex items-center gap-2">
 <Stars score={ownRating.score} />

 {ownRating.score}/5

 </div>
 {ownRating.note ? (
 <p className="mt-2 text-[14px] text-gray-700 leading-relaxed">
 {ownRating.note}
 </p>
) : null}
 <button
 type="button"
 onClick={deleteOwn}
 disabled={submitting}
 className="mt-3 text-[12px] font-medium text-red-600 hover:text-red-800 disabled:opacity-60"
 >
 Delete my rating
 </button>
 {error && (
 <p className="mt-2 text-[12px] text-red-600" role="alert">
 {error}
 </p>
)}
 </div>
) : (
 <div className="rounded-xl border border-gray-200 p-4 space-y-2">
 <p className="text-[13px] text-gray-600">
 Leave a quick rating (and optional tip, 280 chars max):
 </p>
 <Stars score={score} onPick={setScore} />
 <textarea
 maxLength={280}
 rows={2}
 value={note}
 onChange={(e) => setNote(e.target.value)}

```



```
placeholder="Optional ? a quick tip for other shoppers"
className="w-full rounded-lg border border-gray-200 px-3 py-2 text-[13px] focus:outline-none
```

focus:ring

```
/>
{error && (
 <p className="text-[12px] text-red-600" role="alert">
 {error}
 </p>
)}
<div className="flex justify-end">
 <button
 type="button"
 disabled={submitting || score < 1}
 onClick={submit}
 className="px-4 py-1.5 rounded-lg text-[13px] font-semibold bg-emerald-700 text-white hover:bg-
 >
 {submitting ? "Submitting?" : "Submit rating"}
 </button>
</div>
</div>
)
): (
 <div className="rounded-xl border border-dashed border-gray-200 p-3 text-[13px] text-gray-600">
 <Link
 href={shopperLoginHref}
 className="text-emerald-800 font-medium hover:underline"
 >
 Sign in as a shopper
 </Link>{" "}
 to leave a rating.
 </div>
)}
{summary.ratings.length > 0 && (
 <ul className="divide-y divide-gray-100 pt-1">
 {summary.ratings.map((r) => (
 <li key={r.id} className="py-3">
 <div className="flex items-center gap-2">
 <Stars score={r.score} />

 {r.authorName} ? {formatDate(r.createdAt)}

 </div>
 {r.note ? (
 <p className="mt-1 text-[14px] text-gray-700 leading-relaxed">
 {r.note}
 </p>
) : null}

))}

)}
</section>
);
```

emerald-

```
}
```

```
?prisma/migrations/20260419130000_store_ratings/migration.sql?
```

```
+24
```

```
Lines changed: 24 additions & 0 deletions
```

```
Original file line number Diff line number Diff line change
```

```
-- CreateTable
```

```
CREATE TABLE "store_ratings" (
```

```
"id" TEXT NOT NULL,
```

```
"store_id" TEXT NOT NULL,
```

```
"shopper_id" TEXT NOT NULL,
```

```
"score" INTEGER NOT NULL,
```

```
"note" TEXT,
```

```
"created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
"updated_at" TIMESTAMP(3) NOT NULL,
```

```
CONSTRAINT "store_ratings_pkey" PRIMARY KEY ("id")
```

```
);
```

```
-- CreateIndex
```

```
CREATE UNIQUE INDEX "store_ratings_store_id_shopper_id_key" ON "store_ratings"("store_id",
```

```
"shopper
```

```
-- CreateIndex
```

```
CREATE INDEX "store_ratings_store_id_created_at_idx" ON "store_ratings"("store_id", "created_at"
```

```
DESC);
```

```
-- AddForeignKey
```

```
ALTER TABLE "store_ratings" ADD CONSTRAINT "store_ratings_store_id_fkey" FOREIGN KEY ("store_id")
```

```
REFERE
```

```
-- AddForeignKey
```

```
ALTER TABLE "store_ratings" ADD CONSTRAINT "store_ratings_shopper_id_fkey" FOREIGN KEY
```

```
("shoppe
```

```
?prisma/migrations/20260419140000_store_ratings_score_check/migration.sql?
```

```
+4
```

```
Lines changed: 4 additions & 0 deletions
```

```
Original file line number Diff line number Diff line change
```

```
-- Enforce the 1?5 star range at the database layer so a buggy caller
```

```
-- (or direct SQL) can't insert an out-of-range score.
```

```
ALTER TABLE "store_ratings"
```

```
ADD CONSTRAINT "store_ratings_score_range" CHECK ("score" BETWEEN 1 AND 5);
```

```
?prisma/schema.prisma?
```

```
+21
```

```
Lines changed: 21 additions & 0 deletions
```

```
Original file line number Diff line number Diff line change
```

```
shopperNotifications ShopperNotification[]
```

```
profileViews StoreProfileView[]
```

```
itemSearches StoreItemSearch[]
```

```
ratings StoreRating[]
```

```
reports StoreReport[]
```

```
@@map("stores")
```

```
alerts Alert[]
```

```
shopperNotifications ShopperNotification[]
```

```
ratings StoreRating[]
```

```
reports StoreReport[]
```

```
@@map("shoppers")
```

```
@@map("auth_audit_logs")
```

```
}
```

```
/// Story 15: lightweight shopper rating (1?5) + optional tip per store.
```

```

/// Unique per (store, shopper) so a shopper can hold at most one active rating.
model StoreRating {
 id String @id @default(uuid())
 storeId String @map("store_id")
 shopperId String @map("shopper_id")
 score Int
 note String?
 createdAt DateTime @default(now()) @map("created_at")
 updatedAt DateTime @updatedAt @map("updated_at")
 store Store @relation(fields: [storeId], references: [id], onDelete: Cascade)
 shopper Shopper @relation(fields: [shopperId], references: [id], onDelete: Cascade)
 @@unique([storeId, shopperId])
 @@index([storeId, createdAt(sort: Desc)])
 @@map("store_ratings")
}

/// Story 17: shopper-submitted reports of incorrect/outdated store info.
model StoreReport {
 id String @id @default(uuid())
 ?tests/StoreRatingsPanel.test.tsx?
+196
Lines changed: 196 additions & 0 deletions
Original file line number Diff line number Diff line change
/**
 * Story 15 ? Shopper ratings (#119)
 *
 * Covers the ratings panel on the store profile:
 * - Aggregate header renders average + total when ratings exist, and a
 * "no ratings yet" copy otherwise.
 * - Logged-out viewers see the sign-in CTA.
 * - Shoppers without an existing rating can pick stars, submit, and see
 * their rating appear in the "Your rating" card.
 * - Shoppers with an existing rating can delete it, which hits
 * DELETE /api/stores/:id/ratings/:ratingId and restores the form.
 *
 * Tracking issue: #128
 */
import "@testing-library/jest-dom";
import { act, render, screen, waitFor } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import React from "react";
import StoreRatingsPanel from "@components/StoreRatingsPanel";
jest.mock("next/link", () => ({
 __esModule: true,
 default({ children, href }: { children: React.ReactNode; href: string }) {
 return {children};
 },
}));
const EMPTY_SUMMARY = {
 average: null,
 total: 0,
 ratings: [],
 hasMore: false,
};
const POPULATED_SUMMARY = {

```

```

average: 4.3,
total: 12,
ratings: [
{
id: "r-1",
score: 5,
note: "Best produce in Bloomfield.",
createdAt: "2026-04-01T12:00:00.000Z",
authorName: "Alex",
},
],
hasMore: false,
};

describe("StoreRatingsPanel ? aggregate header", () => {
it("shows 'no ratings yet' when average is null", () => {
render(
<StoreRatingsPanel
storeId="s1"
viewerRole={null}
initialSummary={EMPTY_SUMMARY}
initialOwnRating={null}
/>,
);
expect(screen.getByText(/no ratings yet/i)).toBeInTheDocument();
});
it("shows the average and total when ratings exist", () => {
render(
<StoreRatingsPanel
storeId="s1"
viewerRole={null}
initialSummary={POPULATED_SUMMARY}
initialOwnRating={null}
/>,
);
expect(screen.getByText("4.3")).toBeInTheDocument();
expect(screen.getByText(/12 ratings/i)).toBeInTheDocument();
expect(screen.getByText(/best produce in bloomfield/i)).toBeInTheDocument();
});
});
describe("StoreRatingsPanel ? viewer-role gating", () => {
it("shows a sign-in CTA for logged-out viewers", () => {
render(
<StoreRatingsPanel
storeId="s1"
viewerRole={null}
initialSummary={EMPTY_SUMMARY}
initialOwnRating={null}
/>,
);
const link = screen.getByRole("link", { name: /sign in as a shopper/i });
expect(link.getAttribute("href")).toContain("/shopper/login");
});
it("shows the same CTA for owner viewers", () => {
render(

```

```

<StoreRatingsPanel
 storeId="s1"
 viewerRole="owner"
 initialSummary={EMPTY_SUMMARY}
 initialOwnRating={null}
/>,
);
expect(
 screen.getByRole("link", { name: /sign in as a shopper/i }),
).toBeInTheDocument();
});
});
describe("StoreRatingsPanel ? shopper submit flow", () => {
 afterEach(() => {
 jest.restoreAllMocks();
 });
 it("POSTs the selected score and renders 'Your rating' on success", async () => {
 const fetchMock = jest.spyOn(global, "fetch").mockImplementation(
 async (input: RequestInfo | URL, init?: RequestInit) => {
 if (init?.method === "POST") {
 return {
 ok: true,
 json: async () => ({
 rating: { id: "new-1", score: 4, note: "solid" },
 }),
 } as Response;
 }
 return { ok: true, json: async () => POPULATED_SUMMARY } as Response;
 },
);
 render(
 <StoreRatingsPanel
 storeId="s1"
 viewerRole="shopper"
 initialSummary={EMPTY_SUMMARY}
 initialOwnRating={null}
 />,
);
 await userEvent.click(screen.getByRole("button", { name: /4 stars/i }));
 await act(async () => {
 await userEvent.click(
 screen.getByRole("button", { name: /submit rating/i }),
);
 });
 await waitFor(() => {
 expect(screen.getByText(/your rating/i)).toBeInTheDocument();
 });
 expect(screen.getByText(/"4\/5"/)).toBeInTheDocument();
 const postCall = fetchMock.mock.calls.find(
 ([, init]) => (init as RequestInit | undefined)?.method === "POST",
);
 expect(postCall).toBeTruthy();
 expect(postCall?.[0]).toBe("/api/stores/s1/ratings");
 expect(JSON.parse((postCall?.[1] as RequestInit).body as string)).toEqual({

```

```

score: 4,
});
});
it("deletes the shopper's own rating via DELETE", async () => {
const fetchMock = jest.spyOn(global, "fetch").mockImplementation(
async (_input: RequestInfo | URL, init?: RequestInit) => {
if (init?.method === "DELETE") {
return { ok: true, json: async () => ({}) } as Response;
}
return { ok: true, json: async () => EMPTY_SUMMARY } as Response;
},
);
render(
<StoreRatingsPanel
storeId="s1"
viewerRole="shopper"
initialSummary={POPULATED_SUMMARY}
initialOwnRating={{ id: "mine-1", score: 5, note: "love it" }}
/>,
);
expect(screen.getByText(/your rating/i)).toBeInTheDocument();
await act(async () => {
await userEvent.click(
screen.getByRole("button", { name: /delete my rating/i }),
);
});
const deleteCall = fetchMock.mock.calls.find(
([, init]) => (init as RequestInit | undefined)?.method === "DELETE",
);
expect(deleteCall?.[0]).toBe("/api/stores/s1/ratings/mine-1");
await waitFor(() => {
expect(
screen.getByRole("button", { name: /submit rating/i }),
).toBeInTheDocument();
});
});
});
});

```

### --- Turn 131 (assistant) ---

Creating branch `docs/dev-spec-us15` (or similar), reviewing existing dev-spec format, and adding

the US15

[REDACTED]

### --- Turn 132 (assistant) ---

Checking dependency versions and key implementation files for accurate spec content.

[REDACTED]

### --- Turn 133 (assistant) ---

[REDACTED]

### --- Turn 134 (assistant) ---

Creating branch `docs/manual-dev-spec-us15` from `main` and adding `docs/dev-specs/US15-shopper-

store-rati

[REDACTED]

--- Turn 135 (assistant) ---

[REDACTED]

--- Turn 136 (assistant) ---

[REDACTED]

--- Turn 137 (assistant) ---

[REDACTED]

--- Turn 138 (assistant) ---

[REDACTED]

--- Turn 139 (assistant) ---

[REDACTED]

--- Turn 140 (assistant) ---

Writing the spec file with exact required headings.

[REDACTED]

--- Turn 141 (assistant) ---

[REDACTED]

--- Turn 142 (assistant) ---

[REDACTED]

--- Turn 143 (assistant) ---

[REDACTED]

--- Turn 144 (assistant) ---

Here is what was done and how to track it on GitHub.

### Branch and file

- \*\*Branch:\*\* `docs/manual-dev-spec-us15` (created from current `main`, committed, \*\*pushed\*\* to

`origin`).

- \*\*Open a PR:\*\* <https://github.com/gsha22/GrocerEase/pull/new/docs/manual-dev-spec-us15>

- \*\*Spec file:\*\* `docs/dev-specs/US15-shopper-store-ratings.md`

This is the repo's `dev-specs` folder (same tree as `docs/dev-specs/README.md`). A separate root-

level `dev-

The document follows [ `docs/dev-specs/prompts/spec-new.md` ](docs/dev-specs/prompts/spec-new.md):

exactly th

### What a GitHub issue for `create dev specs` should include

Use a short title, for example: \*\*Manual dev spec: US15 shopper store ratings\*\*.

Body checklist:

1. \*\*Goal:\*\* ?Produce a rubric dev spec for US15 (post-merge).?
2. \*\*Links:\*\* User story [#119](https://github.com/gsha22/GrocerEase/issues/119), implementing PR
3. \*\*Deliverable:\*\* Path to the markdown file: `docs/dev-specs/US15-shopper-store-ratings.md` (or
4. \*\*Rubric gate:\*\* Confirm all required `##` headings from `spec-new.md` are present (workflow
5. \*\*Owners:\*\* Confirm primary/secondary in `### Primary and secondary owners` (currently AvalonMei /
6. \*\*Merge date:\*\* If graders need GitHub's exact `merged\_at`, paste it from PR #126 and replace the
7. \*\*Scope note:\*\* Spec is \*\*manual\*\* (not from `dev-spec-on-pr-approval`), so no `dev-spec-

[#126](ht

attach th

validation

gsha22 v

line in `#

If you want the file at the **repository root** in a new `dev-specs/`` directory instead, say so and

[REDACTED]

--- Turn 145 (user) ---

bro the branch cant be called manual-dev-spec-us15, can u just call it dev-spec-us15

--- Turn 146 (assistant) ---

Renaming the branch to `dev-spec-us15`` and updating the remote.

[REDACTED]

--- Turn 147 (assistant) ---

[REDACTED]

--- Turn 148 (assistant) ---

[REDACTED]

--- Turn 149 (assistant) ---

Here is what changed:

- **Local branch** is now `dev-spec-us15`` (the rename from `docs/manual-dev-spec-us15`` completed;

- **Remote:** `origin/dev-spec-us15`` pushed and set as upstream.

- **Old remote** `docs/manual-dev-spec-us15`` was removed so it does not linger.

Open a PR from: <https://github.com/gsha22/GrocerEase/pull/new/dev-spec-us15>

If you still had a PR open against `docs/manual-dev-spec-us15``, GitHub will have closed or broken

[REDACTED]

--- Turn 150 (user) ---

what should i write in the issue body for this dev specs uatation, follow the yml file for this

--- Turn 151 (assistant) ---

Reading the workflow and prompt YAML to list what the automation consumes and what an issue body

[REDACTED]

--- Turn 152 (assistant) ---

[REDACTED]

--- Turn 153 (assistant) ---

The workflow **does not** read a random GitHub issue's body to decide whether to run. It runs on

So treat the ?issue body? as either **(A)** a team process issue you file for documentation, or

---

### Paste this into a GitHub issue (process / ?how dev spec automation works?)

```markdown

Dev spec automation (repo workflow)

This repo generates CMU-style development specs using **GitHub Actions** + **GitHub Models**

- `.github/workflows/dev-spec-on-pr-approval.yml`
- `.github/prompts/dev-spec-generate.prompt.yml`

When it runs

- **Event:** Someone submits a **pull request review** with state `approved`.
- **PR must:** target branch `main`, and still be `open` (approval before merge).
- **If those are not true**, the job does not run (or has nothing useful to do).

How the workflow picks the story slug (`USn`)

The workflow parses **the PR title and PR body** (concatenated) with a regex. It recognizes patterns

- `US4`, `US 4`, `us 15`, `Story 4`, `story: 4`, etc. (story / US + 1?3 digits)

If nothing matches, it falls back to a PR **label** whose name starts with:

- `dev-spec:USn` (e.g. `dev-spec:US15`) ? the part after the colon is normalized to `US` + digits

Action for contributors: Put `US15` (or `Story 15`, etc.) in the **PR title or PR body**, or

What gets sent to the model (from the YAML)

The prompt is built from:

- | Input | Source |
- |-----|-----|
- | PR context | PR **title** + **body** ? `context.md` |
- | Story slug | Parsed as above ? `story_slug.md` |
- | Issue links | **Closes** / **Fixes** / **Resolves** / **Refs** `#123` in the **PR body** only ?
- | Owners | PR **author** and **assignees** ? `owners_context.md` |
- | Merge state | `merged`, `merged_at`, review `submitted_at` ? `merge_context.md` |
- | Changed files | `git diff --name-only` (with excludes) ? `changed_files.md` |
- | Instructions | `docs/dev-specs/prompts/spec-new.md` or `spec-update.md` if a prior spec exists on
- | Diff | Unified diff (capped size) ? `diff.txt` |

So **Closes #119** style lines belong on the **pull request** description if you want those links

Output file on the docs branch

Generated file path (from the workflow):

- `docs/dev-specs/<SLUG>-development-spec.md`
- Example: slug `US15` ? `docs/dev-specs/US15-development-spec.md`

The model must follow the exact `##` headings in `docs/dev-specs/prompts/spec-new.md` (validated in

After merge

If graders need an exact merge timestamp: the prompt uses `pull_request.merged_at` when present; if

Permissions / infra

Workflow uses `GITHUB_TOKEN` with `contents`, `pull-requests`, `issues`, and `models: read` (see

...

What you must put on the **feature PR** (this is what the YAML actually reads)

1. **Title or body:** `US15` / `Story 15` / etc., **or** label `dev-spec:US15`.
2. **Body:** `Closes #119` (or `Refs #?`) so **issue_links** in the prompt lists the right issues.
3. **Assignees** (optional but helpful): populate PR assignees so **primary/secondary owners** in
4. **Approve** the PR **into** `main` while it is still open to trigger generation (then merge the

A standalone issue body **does not** trigger or feed this workflow unless the same text also appears

[REDACTED]

--- **Turn 154 (user)** ---

US15: Dev Specs Automation #137

can you add that the issue tracking this dev specs automation is issue #137

--- **Turn 155 (assistant)** ---

[REDACTED]

--- **Turn 156 (assistant)** ---

[REDACTED]

--- **Turn 157 (assistant)** ---

Adding issue #137 to `docs/dev-specs/README.md` and a short reference in the workflow file header.

[REDACTED]

--- **Turn 158 (assistant)** ---

Here is what was added.

`docs/dev-specs/README.md`

- New section **## Automation meta tracking** with a link to [issue

- Under **## Tracking issue**, one sentence clarifies that those issues are per story/PR, distinct

`.github/workflows/dev-spec-on-pr-approval.yml`

- Header comment:

`# Meta tracking for this automation: <https://github.com/gsha22/GrocerEase/issues/137>`

Commit on your branch when ready, for example:

```
```bash
git add docs/dev-specs/README.md .github/workflows/dev-spec-on-pr-approval.yml
git commit -m "docs(dev-specs): link automation meta tracking to issue #137"
git push
```
```

[REDACTED]

--- **Turn 159 (user)** ---

what should be in the body of the pr

--- **Turn 160 (assistant)** ---

It depends which PR you mean.

Feature PR (the one that **triggers** dev-spec automation)

The workflow only reads **this PR's** title/body (plus labels, assignees, merge fields). Put in the

1. **Story marker** (if not already obvious in the title), e.g. `Implements **US15**` or `Story 15`
2. **Linked issues** using GitHub keywords so they show up in `issue\_links` for the model, e.g. `Closes #119` or `Fixes #119` / `Refs #137` as appropriate.
 - Use **Closes` / `Fixes` / `Resolves`** for the user story you're finishing.
 - Use **Refs #137`** if you want the automation prompt to mention the dev-spec automation tracking
3. **Short summary** for humans and the LLM: what changed, main files, any caveats.
4. **How to test** (commands or manual steps).
5. **Screenshots** if UI changed (optional but helpful).

Example skeleton:

```
```markdown
Summary
?

User story
US15 ? ?

Closes #119
Refs #137

Test plan
- [] ?
```
```

Note: Automation runs on **approval** of a PR **into `main`** while the PR is **open**. After

Docs PR (e.g. manual `US15-shopper-store-ratings.md` + README #137)

Same idea, but **slug in title/body** only matters if you care about humans; the **approval**

Use:

- What changed and why.
- `Refs #119` / `Refs #137` (or `Closes` only if this PR actually closes those issues?usually

If you tell me whether this PR is **feature** vs **docs-only**, I can tighten it to one copy-paste

[REDACTED]

--- Turn 161 (user) ---

great now i need to go generate the same report with similar structure with us16:

here is the original issue for us16:

US 16: Map Filter and Map UI Improvement #120

Closed

0 / 1

0 of 1 issue completed

#124

Closed

US 16: Map Filter and Map UI Improvement

#120

0 / 1

0 of 1 issue completed

#124

@AvalonMei

Description

AvalonMei

opened 3 days ago

As a shopper, I want to filter the map and store list by specific categories (e.g., "Asian

Specialty

Why This Is Important

Shoppers with specific needs (e.g., looking for fresh produce or specialty items) currently have no

way to na

Human Acceptance Criteria

Shoppers see a row of category buttons/tags at the top of the store discovery page

Clicking a category (e.g., "Produce") instantly filters the map pins and store cards to show only

matching

Each store card clearly displays its primary category tags

Machine Acceptance Criteria

The database query for stores supports an optional category parameter

The API returns an empty list (200 OK) if no stores match the selected category filter

Filtering logic is handled client-side or server-side such that the UI updates in under 300ms

the pr that closed it:

US16: Add category filters to Map view (#120)

#124

Merged

AvalonMei

merged 4 commits into

main

from

feature/us-16-map-filters

yesterday

+245

-19

Lines changed: 245 additions & 19 deletions

Conversation6 (6)

Commits4 (4)

Checks5 (5)

Files changed2 (2)

Merged

US16: Add category filters to Map view (#120)#124

AvalonMei

merged 4 commits into

main

from

feature/us-16-map-filters

Conversation

@gsha22

gsha22

commented

yesterday

Closes #120

Summary

Reuses the existing StoreFilterBar (already on /) on /map, wires the selected categories to the existing /api/stores?category= param (AND logic, case-insensitive ? already supported by the backend), shows up to three category tags on each store card under the map, and updates the header count / empty-state copy when filters return no results.

Machine acceptance criteria (from issue)

/api/stores supports an optional category parameter ? already shipped; now surfaced on map view.

API returns [] (200 OK) when no stores match the selected filter.

Filter toggles update the UI in well under 300 ms (single client-side fetch, no navigation).

Human acceptance criteria

Category buttons at the top of the map discovery page.

Clicking a category instantly filters map pins + store cards.

Each store card shows its primary category tags.

Automation note

Implemented via Claude Code (Opus 4.7) as part of P4 task 3. Prompt/procedure documented in the P4

PDF.

? Generated with Claude Code

@gsha22

WIP: US16 map category filters (issue #120)

42d83c8

@github-actions

github-actions bot

commented

yesterday

Summary

This PR adds category filtering to /map, wiring StoreFilterBar into /api/stores?category= and

surfacing

Issues

Severity File Finding

WARN app/(public)/map/page.tsx (L80, L83, L86) Geolocation callbacks capture stale

activeFilt

Action items

Use latest filters in geolocation callbacks (dependency-safe effect or useRef for current filters).

Ensure initial geolocation response cannot overwrite a newer filter-triggered fetch.

Automated review: GitHub Copilot CLI in GitHub Actions.

_Workflow: Copilot PR review ? Run: <https://github.com/gsha22/GrocerEase/actions/runs/24636776779>

@gsha22 gsha22 mentioned this pull request yesterday

docs: P4 code-generation procedure #127

Merged

@gsha22

fix(us16): preserve "Finding stores near you" copy for empty default ?

036b33c

@github-actions

github-actions bot

commented

yesterday

Summary

This PR successfully adds category filtering to the map page and surfaces category tags on store

cards. Th

Issues

Severity File Finding

BLOCKER app/(public)/map/page.tsx (L44?79) Concurrent fetches can resolve out-of-order,

showing

WARN app/(public)/map/page.tsx (L66?79) Geolocation callback uses initial activeFilters; late

resolve o

Action items

Add request cancellation/ordering (AbortController or request ID guard) in loadStores.

Ensure geolocation-triggered load reads latest filters (ref/deps), not initial closure.

Prevent older responses from calling setStores after newer filter changes.

Automated review: GitHub Copilot CLI in GitHub Actions.

_Workflow: Copilot PR review ? Run: <https://github.com/gsha22/GrocerEase/actions/runs/24637305356>

gsha22 added a commit that referenced this pull request yesterday

@gsha22

docs: add System prompt used for debugging ?

ad23b3f

@gsha22

test(us16): Jest coverage for category filter wiring

71772f4

@github-actions

github-actions bot

commented

yesterday

Summary

This PR adds category filtering to /map, wiring StoreFilterBar to /api/stores?category=..., updating

empty-st

Issues

Severity File Finding

BLOCKER app/(public)/map/page.tsx (useEffect, ~68-90) Geolocation callback uses stale

activeFilt

WARN app/(public)/map/page.tsx (loadStores, handleFiltersChange) No request

ordering/

Action items

Make initial geolocation load use current filters (e.g., read latest via ref/effect deps).

Guard against stale responses (AbortController or request-id check before setStores).

Automated review: GitHub Copilot CLI in GitHub Actions.

_Workflow: Copilot PR review ? Run: <https://github.com/gsha22/GrocerEase/actions/runs/24637863084>

@AvalonMei

AvalonMei

commented

yesterday

Can you fix the issues brought up by Copilot? Otherwise everything lgtm.

@gsha22

fix(us16): request-id guard + filter ref to prevent stale-fetch clobber

2a71f21

@github-actions

github-actions bot

commented

yesterday

Summary

This PR adds category filtering to the map discovery page by wiring StoreFilterBar into /map and

passing s

Issues

No issues found.

Action items

None blocking.

Automated review: GitHub Copilot CLI in GitHub Actions.

_Workflow: Copilot PR review ? Run: <https://github.com/gsha22/GrocerEase/actions/runs/24638432292>

@AvalonMei

AvalonMei

commented

yesterday

LGTM

@AvalonMei AvalonMei merged commit 889bede into main yesterday

5 checks passed

files changed:

US16: Add category filters to Map view (#120)

#124

Merged

AvalonMei

merged 4 commits into

main

from

feature/us-16-map-filters

yesterday

+245

-19

Lines changed: 245 additions & 19 deletions

Conversation6 (6)

Commits4 (4)

Checks5 (5)

Files changed2 (2)

Pull Request Toolbar

Merged

US16: Add category filters to Map view (#120)

#124

AvalonMei

merged 4 commits into

main

from

feature/us-16-map-filters

0 / 2 viewed

Filter files?

File tree

app/(public)/map

page.tsx

tests

MapPageCategoryFilters.test.tsx

?app/(public)/map/page.tsx?

+74

-19

Lines changed: 74 additions & 19 deletions

Original file line number Diff line number Diff line change

```

// Story 3: Discover Nearby Stores ? Map View
// Story 12: No auth required
// Story 16: Category filter on map view (reuses /api/stores?category=)
"use client";

import { useEffect, useState } from "react";
import { useCallback, useEffect, useRef, useState } from "react";
import dynamic from "next/dynamic";
import Link from "next/link";
import StoreFilterBar, { type FilterKey } from "@/components/StoreFilterBar";

// Leaflet requires window ? load only on client
const StoreMap = dynamic(() => import("@/components/StoreMap"), { ssr: false });
export function buildMapStoresApiUrl(
  lat: number | null,
  lng: number | null,
  categories: Set<FilterKey> = new Set(),
): string {
  const params = new URLSearchParams();
  if (lat !== null && lng !== null) {
    params.set("lat", lat.toString());
    params.set("lng", lng.toString());
    params.set("radius", "10");
  }
  if (categories.size > 0) {
    params.set("category", Array.from(categories).join(", "));
  }
  return `/api/stores${params.toString() ? `?${params}` : ``}`;
}

export default function MapPage() {
  const [stores, setStores] = useState<Store[]>([]);
  const [center, setCenter] = useState<[number, number]>(PITTSBURGH_CENTER);
  const [loading, setLoading] = useState(true);
  const [coords, setCoords] = useState<{ lat: number; lng: number } | null>(
    null,
  );
  const [activeFilters, setActiveFilters] = useState<Set<FilterKey>>(new Set());

  useEffect(() => {
    async function loadStores(lat: number | null, lng: number | null) {
      const res = await fetch(buildMapStoresApiUrl(lat, lng));
      const data = await res.json();
      setStores(data);
      setLoading(false);
    }

    // Request-ordering guard: only the most recent `loadStores` call is
    // allowed to write to state. Avoids the race where a slow initial
    // geolocation fetch clobbers a newer filter-triggered result.
    const latestRequestId = useRef(0);
    const activeFiltersRef = useRef(activeFilters);
    activeFiltersRef.current = activeFilters;
    const loadStores = useCallback(
      async (
        lat: number | null,
        lng: number | null,

```



```

filters: Set<FilterKey>,
) => {
  const requestId = ++latestRequestId.current;
  setLoading(true);
  try {
    const res = await fetch(buildMapStoresApiUrl(lat, lng, filters));
    const data = await res.json();
    if (requestId !== latestRequestId.current) return;
    setStores(data);
  } finally {
    if (requestId === latestRequestId.current) setLoading(false);
  }
},
[],
);

useEffect(() => {
  // Always read the *current* filters via ref ? the geolocation
  // callback may resolve after the user has already selected filters.
  if ("geolocation" in navigator) {
    navigator.geolocation.getCurrentPosition(
      (pos) => {
        const loc: [number, number] = [
          pos.coords.latitude,
          pos.coords.longitude,
        ];
        setCenter(loc);
        loadStores(loc[0], loc[1]);
        const loc = {
          lat: pos.coords.latitude,
          lng: pos.coords.longitude,
        };
        setCoords(loc);
        setCenter([loc.lat, loc.lng]);
        loadStores(loc.lat, loc.lng, activeFiltersRef.current);
      },
      () => loadStores(null, null)
    );
    () => loadStores(null, null, activeFiltersRef.current),
  );
} else {
  loadStores(null, null);
  loadStores(null, null, activeFiltersRef.current);
}
}, []);
}, [loadStores]);

function handleFiltersChange(next: Set<FilterKey>) {
  setActiveFilters(next);
  loadStores(coords?.lat ?? null, coords?.lng ?? null, next);
}

return (
  <div className="max-w-[1100px] mx-auto px-4 sm:px-6 lg:px-8 py-8">
    </h1>
    <p className="text-[15px] text-gray-600 mt-1.5">
      {stores.length > 0

```

```

? `${stores.length} stores within 10 miles`
: "Finding stores near you?"}
? `${stores.length} store${stores.length === 1 ? "" : "s"} within 10 miles`
: !loading && activeFilters.size > 0
? "No stores match the selected filters."
: "Finding stores near you?"}
</p>
</div>

{/* Category filters ? Story 16 */}
<StoreFilterBar active={activeFilters} onChange={handleFiltersChange} />
{/* View toggle */}
<div className="flex border border-gray-200 rounded-md overflow-hidden w-fit mb-5">
<Link
<div className="text-[12px] text-gray-400 mt-0.5 truncate">
{store.address}
</div>
{store.categories.length > 0 && (
<div className="flex flex-wrap gap-1 mt-1.5">
{store.categories.slice(0, 3).map((tag) => (
<span
key={tag}
className="text-[10px] px-1.5 py-0.5 rounded-full bg-emerald-50 text-emerald-800 font-medium"
>
{tag}
</span>
)}}
</div>
)}
</div>
{store.distanceMiles !== null && (
<span className="text-[13px] text-green-600 font-medium shrink-0">
?tests/MapPageCategoryFilters.test.tsx?
+171
Lines changed: 171 additions & 0 deletions
Original file line number   Diff line number   Diff line change
/**
 * Story 16 ? Map category filters (#120)
 *
 * Covers the UI + URL-wiring contract of the filter bar on /map:
 * - buildMapStoresApiUrl serializes selected categories as a comma-separated
 *   `category=` query param and omits the param when nothing is selected.
 * - The map page renders the filter chips and the store-card category tags.
 * - Toggling a chip refetches /api/stores with the category param.
 * - The empty header copy distinguishes "no filters at all" from
 *   "filters selected but no match".
 *
 * Tracking issue: #129
 */
import "@testing-library/jest-dom";
import { act, render, screen, waitFor } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import React from "react";
import MapPage, { buildMapStoresApiUrl } from "@app/(public)/map/page";

```

```

jest.mock("next/dynamic", () => ({
  __esModule: true,
  default: () =>
    function FakeMap() {
      return <div data-testid="store-map">map</div>;
    },
}));
jest.mock("next/link", () => ({
  __esModule: true,
  default({ children, href }: { children: React.ReactNode; href: string }) {
    return <a href={href}>{children}</a>;
  },
}));
function mockGeolocationSuccess() {
  Object.defineProperty(navigator, "geolocation", {
    configurable: true,
    value: {
      getCurrentPosition: jest.fn((success: PositionCallback) => {
        success({
          coords: { latitude: 40.5, longitude: -80.1 },
        } as GeolocationPosition);
      }),
    },
  });
}
const ASIAN_STORE = {
  id: "asian-1",
  name: "Lotus Asian Market",
  address: "100 Penn Ave",
  lat: 40.5,
  lng: -80.1,
  categories: ["asian"],
  distanceMiles: 0.5,
};
describe("buildMapStoresApiUrl ? categories", () => {
  it("omits the category param when none are selected", () => {
    expect(buildMapStoresApiUrl(40.44, -79.99, new Set())).not.toContain(
      "category=",
    );
  });
  it("joins multiple categories with commas", () => {
    const url = buildMapStoresApiUrl(
      40.44,
      -79.99,
      new Set(["asian", "halal"]) as Set<"asian" | "halal">,
    );
    expect(url).toContain("category=asian%2Chalal");
  });
  it("serializes category only, even without coords", () => {
    const url = buildMapStoresApiUrl(
      null,
      null,
      new Set(["produce"]) as Set<"produce">,
    );
  });

```

```

expect(url).toBe("/api/stores?category=produce");
});
});
describe("MapPage ? category filter UI", () => {
  beforeEach(() => {
    mockGeolocationSuccess();
    jest.spyOn(global, "fetch").mockResolvedValue({
      ok: true,
      json: async () => [ASIAN_STORE],
    } as Response);
  });
  afterEach(() => {
    jest.restoreAllMocks();
  });
  it("renders a row of category filter chips", async () => {
    render(<MapPage />);
    await waitFor(() =>
      expect(screen.getByTestId("store-map")).toBeInTheDocument(),
    );
    // At minimum the four primary discovery categories from the issue.
    expect(
      screen.getByRole("button", { name: /asian/i }),
    ).toBeInTheDocument();
    expect(
      screen.getByRole("button", { name: /halal/i }),
    ).toBeInTheDocument();
    expect(
      screen.getByRole("button", { name: /produce/i }),
    ).toBeInTheDocument();
  });
  it("shows category tags on each store card under the map", async () => {
    render(<MapPage />);
    await waitFor(() => {
      expect(screen.getByText("Lotus Asian Market")).toBeInTheDocument();
    });
    // The card surfaces the 'asian' tag from ASIAN_STORE.categories
    expect(screen.getAllByText("asian").length).toBeGreaterThan(0);
  });
  it("refetches /api/stores with category=asian when the Asian chip is toggled", async () => {
    const fetchMock = global.fetch as jest.Mock;
    render(<MapPage />);
    await waitFor(() =>
      expect(screen.getByTestId("store-map")).toBeInTheDocument(),
    );
    fetchMock.mockClear();
    await act(async () => {
      await userEvent.click(screen.getByRole("button", { name: /asian/i }));
    });
    await waitFor(() => {
      expect(fetchMock).toHaveBeenCalledWith(
        expect.stringContaining("category=asian"),
      );
    });
  });
});

```

```
it("shows the filtered empty-state copy when no stores match", async () => {
  const fetchMock = global.fetch as jest.Mock;
  // First call returns one store (initial load); after filter selection,
  // return an empty array to simulate "no matches".
  fetchMock
    .mockResolvedValueOnce({
      ok: true,
      json: async () => [ASIAN_STORE],
    } as Response)
    .mockResolvedValue({
      ok: true,
      json: async () => [],
    } as Response);
  render(<MapPage />);
  await waitFor(() =>
    expect(screen.getByText("Lotus Asian Market")).toBeInTheDocument(),
  );
  await act(async () => {
    await userEvent.click(screen.getByRole("button", { name: /halal/i }));
  });
  await waitFor(() => {
    expect(
      screen.getByText(/No stores match the selected filters/i),
    ).toBeInTheDocument();
  });
});
```

here is the dev-spec-tracking issue: US16 : Dev Specs Automation #139

--- Turn 162 (assistant) ---

Creating a US16 dev spec matching the US15 rubric. Inspecting the implementation and merge metadata.

[REDACTED]

--- Turn 163 (assistant) ---

[REDACTED]

--- Turn 164 (assistant) ---

[REDACTED]

--- Turn 165 (assistant) ---

[REDACTED]

--- Turn 166 (assistant) ---

[REDACTED]

--- Turn 167 (assistant) ---

[REDACTED]

--- Turn 168 (assistant) ---

Writing `docs/dev-specs/US16-map-category-filters.md` using the US15 rubric structure and

implemen

[REDACTED]

--- Turn 169 (assistant) ---

[REDACTED]

--- Turn 170 (assistant) ---

[REDACTED]

--- Turn 171 (assistant) ---

Creating branch `dev-spec-us16` from `main` so the US16 spec can ship in its own PR; committing the

[REDACTED]

--- Turn 172 (assistant) ---

[REDACTED]

--- Turn 173 (assistant) ---

Here is what was added and how to use it.

New file

docs/dev-specs/US16-map-category-filters.md ? same

It is grounded in:

- PR [#124](https://github.com/gsha22/GrocerEase/pull/124), merge commit `889bede` ?

- Issue [#120](https://github.com/gsha22/GrocerEase/issues/120) (closed by that PR)

- Automation tracking [#139](https://github.com/gsha22/GrocerEase/issues/139) in **## Merge date**

Branch / PR

- **Branch:** `dev-spec-us16` (from current `main`, **only** adds the US16 spec file).

- **Open PR:** https://github.com/gsha22/GrocerEase/pull/new/dev-spec-us16

Suggested PR body:

``markdown

Summary

Adds a manual development specification for **US16** (map category filters / UI), matching the

Links

- Closes: (leave empty unless you want GitHub to close an issue)

- Refs #120

- Refs #139

Notes

- File: `docs/dev-specs/US16-map-category-filters.md`

- Automated workflow would write `docs/dev-specs/US16-development-spec.md`; this manual doc uses a

...

Your older **dev-spec-us15** branch is unchanged locally if you still have it; **dev-spec-us16**

[REDACTED]

--- Turn 174 (user) ---

ok do the same thing for us17:

US 17: Shopper Reports Incorrect Information #121

Closed

#125

Closed

US 17: Shopper Reports Incorrect Information

#121

#125

@AvalonMei

Description

AvalonMei

opened 3 days ago

As a shopper, I want to report incorrect or outdated store information so that I can help keep the

platform

Why This Is Important

The entire product depends on trust ? a single wasted trip due to bad info causes user churn.

Currently

Human Acceptance Criteria

A shopper can submit a report in under 15 seconds

The reporting UI feels lightweight, not punitive

Reports feel acknowledged (confirmation message shown)

Machine Acceptance Criteria

POST /stores/:id/report creates a report with:

type (one of: out_of_stock, incorrect_hours, wrong_price, other)

optional comment

Reports are stored with user_id and store_id

Duplicate reports from the same user within 24h are blocked

Store owners can view reports via /dashboard/reports

the pr that closed it:

Skip to content

gsha22

GrocerEase

Repository navigation

Code

Issues

9

(9)

Pull requests

2

(2)

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

US17: Shopper reports for incorrect store info (#121)

#125

Merged

AvalonMei

merged 3 commits into

main

from

feature/us-17-store-reports

yesterday

+632

Lines changed: 632 additions & 0 deletions

Conversation5 (5)
Commits3 (3)
Checks5 (5)
Files changed9 (9)
Conversation
@gsha22
gsha22
commented
yesterday
Closes #121

Summary

Adds a lightweight way for shoppers to flag incorrect/outdated store info so owners can fix it.

Prisma model StoreReport + enum StoreReportType (out_of_stock, incorrect_hours, wrong_price, other) + migration.

POST /api/stores/[id]/report ? shopper auth required, 280-char comment cap, 24-hour duplicate guard per (shopper, store) returning 429 (spec says "blocked"; we surface an actionable status rather than 409 since this is rate-limit-shaped).

GET /api/stores/[id]/reports ? owner-only, own-store-only.

StoreReportButton client island on /stores/[id]: type chips + optional comment, in-place acknowledgement.

/dashboard/reports page + sidebar entry for owners.

Machine acceptance criteria

POST /stores/:id/report stores {type, comment?, user_id, store_id}.

Duplicate reports from the same user within 24h are blocked.

Owners can view reports via /dashboard/reports.

Human acceptance criteria

Submit in under 15 seconds (single inline form, 4 type chips).

Lightweight, not punitive tone.

Confirmation message after submission.

Automation note

Implemented via Claude Code (Opus 4.7) as part of P4 task 3. Prompt/procedure documented in the P4

PDF.

? Generated with Claude Code

@gsha22

US17: Shopper reports for incorrect store info (closes #121)

ff54ef7

@github-actions

github-actions bot

commented

yesterday

Summary

This PR adds end-to-end shopper reporting for incorrect store info, including schema, APIs, UI entry

point, and

Issues

Severity File Finding

WARN app/api/stores/[id]/report/route.ts L58-82 check-then-create race can allow duplicate

reports w

Action items

Make duplicate blocking atomic: add DB-enforced window guard or transaction/locking strategy.

Automated review: GitHub Copilot CLI in GitHub Actions.

_Workflow: Copilot PR review ? Run: <https://github.com/gsha22/GrocerEase/actions/runs/24636908596>

@gsha22 gsha22 mentioned this pull request yesterday

docs: P4 code-generation procedure #127

Merged

@gsha22

test(us17): Jest coverage for shopper report flow

aafe843

@github-actions

github-actions bot

commented

yesterday

Summary

This PR adds end-to-end shopper reporting for incorrect store info: new StoreReport

schema/

Issues

Severity File Finding

WARN app/api/stores/[id]/report/route.ts (around L54-L84) 24h dedupe is race-prone; concurrent

POSTs o

WARN app/(dashboard)/dashboard/reports/page.tsx (around L22-L49) No owner-role gate; shoppers

can acce

Action items

Make dedupe atomic (transaction/locking or persisted dedupe key strategy) in POST

/api/store

Enforce owner-only access on /dashboard/reports (e.g., requireOwnerSession or redirect/403).

Automated review: GitHub Copilot CLI in GitHub Actions.

_Workflow: Copilot PR review ? Run: <https://github.com/gsha22/GrocerEase/actions/runs/24637881437>

@AvalonMei

AvalonMei

commented

yesterday

Can you fix the 2 issues brought up by Copilot?

@gsha22

fix(us17): atomic 24h dedupe + owner-role gate on /dashboard/reports

5f70bb2

@github-actions

github-actions bot

commented

yesterday

Summary

This PR adds end-to-end shopper reporting for incorrect store info: new StoreReport persistence,

shopper

Issues

No issues found.

Action items

None blocking.

Automated review: GitHub Copilot CLI in GitHub Actions.

_Workflow: Copilot PR review ? Run: <https://github.com/gsha22/GrocerEase/actions/runs/24638462489>

@AvalonMei

AvalonMei

commented

yesterday

LGTM

@AvalonMei AvalonMei merged commit d1d3c08 into main yesterday

5 checks passed

@evelyn-lo evelyn-lo mentioned this pull request now

US17: Dev Specs Automation #141

Open

Merge info

Pull request successfully merged and closed

You're all set ? the feature/us-17-store-reports branch can be safely deleted.

@evelyn-lo

Add a comment

Comment

Add your comment here...

Remember, contributions to this repository should follow our GitHub Community Guidelines.

ProTip! Add .patch or .diff to the end of URLs for Git?s plaintext views.

Reviewers

No reviews

Assignees

No one?

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

US 17: Shopper Reports Incorrect Information

Notifications

Customize

You're not receiving notifications from this thread.

2 participants

@gsha22

@AvalonMei

Footer

? 2026 GitHub, Inc.

Footer navigation

Terms

Privacy

Security

Status

Community

Docs

Contact

Manage cookies

Do not share my personal information

files changed:

Skip to content

gsha22

GrocerEase

Repository navigation

Code

Issues

9

(9)

Pull requests

2

(2)

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

US17: Shopper reports for incorrect store info (#121)

#125

Merged

AvalonMei

merged 3 commits into

main

from

feature/us-17-store-reports

yesterday

+632

Lines changed: 632 additions & 0 deletions

Conversation5 (5)

Commits3 (3)

Checks5 (5)

Files changed9 (9)

Pull Request Toolbar

Merged

US17: Shopper reports for incorrect store info (#121)

#125

AvalonMei

merged 3 commits into

main

from

feature/us-17-store-reports

0 / 9 viewed

Filter files?

File tree

app

(dashboard)/dashboard/reports

page.tsx

(public)/stores/[id]

page.tsx

api/stores/[id]

report

route.ts

reports

route.ts

components

Sidebar.tsx

StoreReportButton.tsx

prisma

migrations/20260419120000_store_reports

migration.sql

schema.prisma

tests

StoreReportButton.test.tsx

?app/(dashboard)/dashboard/reports/page.tsx?

+118

Lines changed: 118 additions & 0 deletions

Original file line number Diff line number Diff line change

// Story 17: Owner view of shopper-submitted reports for their own store.

import Link from "next/link";

import { redirect } from "next/navigation";

import { auth } from "@/auth";

import { prisma } from "@/lib/prisma";

export const dynamic = "force-dynamic";

const TYPE_LABELS: Record<string, string> = {

 out_of_stock: "Out of stock",

 incorrect_hours: "Hours wrong",

 wrong_price: "Price wrong",

 other: "Other",

};

function formatReportTime(d: Date) {

 return new Intl.DateTimeFormat("en-US", {

 month: "short",

 day: "numeric",

 hour: "numeric",

 minute: "2-digit",

 }).format(d);

}

export default async function DashboardReportsPage() {

 const session = await auth();

 if (!session?.user?.id || session.role !== "owner") {

 redirect("/login");

 }

 const ownerId = session.user.id;

 const store = await prisma.store.findUnique({

 where: { ownerId },

 select: { id: true, name: true },

 });

 if (!store) {

 return (

 <div className="mx-auto max-w-3xl px-4 py-10">

 <h1 className="font-display text-2xl font-semibold text-gray-800">

Reports

 </h1>

 <p className="mt-4 text-[14px] text-gray-500">

 Create a store profile first to start receiving shopper reports.

 </p>

 <Link

 href="/dashboard/profile"

 className="mt-4 inline-block text-emerald-800 font-medium hover:underline"

```

>
Go to store profile ?
</Link>
</div>
);
}

const reports = await prisma.storeReport.findMany({
  where: { storeId: store.id },
  orderBy: { createdAt: "desc" },
  select: {
    id: true,
    type: true,
    comment: true,
    createdAt: true,
  },
  take: 100,
});

return (
  <div className="mx-auto max-w-3xl px-4 py-10">
    <div className="mb-6">
      <h1 className="font-display text-2xl font-semibold text-gray-800">
        Reports
      </h1>
      <p className="mt-1 text-[14px] text-gray-500">
        Shoppers have flagged these about <strong>{store.name}</strong>. Use
        them as hints ? they are anonymous.
      </p>
    </div>
    {reports.length === 0 ? (
      <div className="rounded-2xl border border-dashed border-gray-200 bg-white px-6 py-12 text-center">
        <p className="text-[15px] font-medium text-gray-700">No reports yet</p>
        <p className="mt-2 text-[13px] text-gray-500">
          When a shopper reports incorrect info for your store, it will appear
          here.
        </p>
      </div>
    ) : (
      <ul className="space-y-2">
        {reports.map((r) => (
          <li
            key={r.id}
            className="rounded-xl border border-gray-200 bg-white p-4"
          >
            <div className="flex items-start justify-between gap-3">
              <span className="inline-flex items-center rounded-full bg-emerald-50 px-2.5 py-0.5 text-[12px] font-
                {TYPE_LABELS[r.type] ?? r.type}
              </span>
              <span className="text-[12px] text-gray-400 shrink-0">
                {formatReportTime(r.createdAt)}
              </span>
            </div>
            {r.comment ? (
              <p className="mt-2 text-[14px] text-gray-700 leading-relaxed">

```

semibold

```

    {r.comment}
  </p>
) : (
  <p className="mt-2 text-[13px] text-gray-400 italic">
    No additional comment.
  </p>
)}
</li>
)}}
</ul>
)}
</div>
);
}

```

?app/(public)/stores/[id]/page.tsx?

+6

Lines changed: 6 additions & 0 deletions

Original file line number Diff line number Diff line change

```

import ItemAvailabilitySearch from "@components/ItemAvailabilitySearch";
import StoreAlertSubscribe from "@components/StoreAlertSubscribe";
import StoreFreshUpdatesFeed from "@components/StoreFreshUpdatesFeed";
import StoreReportButton from "@components/StoreReportButton";

export const dynamic = "force-dynamic";

/>
</div>

```

{/* Report incorrect info ? Story 17 */}

```

<div className="mt-6 mb-4">
  <StoreReportButton storeId={store.id} viewerRole={viewerRole} />
</div>

```

{/* Deals This Week ? Story 2 */}

```

<div className="bg-white rounded-2xl border border-gray-200 p-6">
  <h2 className="text-[17px] font-semibold text-gray-800 mb-4 flex items-center gap-2">
    ?app/api/stores/[id]/report/route.ts?
  +120

```

Lines changed: 120 additions & 0 deletions

Original file line number Diff line number Diff line change

// Story 17: Shopper reports of incorrect or outdated store information.

```

import { NextRequest, NextResponse } from "next/server";
import { prisma } from "@lib/prisma";
import { requireShopperSession } from "@lib/require-shopper-session";

const REPORT_TYPES = [
  "out_of_stock",
  "incorrect_hours",
  "wrong_price",
  "other",
] as const;

type ReportType = (typeof REPORT_TYPES)[number];

const COMMENT_MAX = 280;
const DUPLICATE_WINDOW_MS = 24 * 60 * 60 * 1000;

export async function POST(
  req: NextRequest,
  ctx: { params: Promise<{ id: string }> },
) {

```

```

const gate = await requireShopperSession();
if (!gate.ok) return gate.response;
const { session } = gate;
const { id: storeId } = await ctx.params;
const body = await req.json().catch(() => null);
const type = body?.type;
const commentRaw = body?.comment;
if (!type || !REPORT_TYPES.includes(type)) {
  return NextResponse.json(
    { error: "Invalid report type", allowed: REPORT_TYPES },
    { status: 400 },
  );
}
let comment: string | null = null;
if (commentRaw != null) {
  if (typeof commentRaw !== "string") {
    return NextResponse.json(
      { error: "comment must be a string" },
      { status: 400 },
    );
  }
  const trimmed = commentRaw.trim();
  if (trimmed.length > COMMENT_MAX) {
    return NextResponse.json(
      { error: `comment too long (max ${COMMENT_MAX} characters)` },
      { status: 400 },
    );
  }
  comment = trimmed || null;
}
const store = await prisma.store.findUnique({
  where: { id: storeId },
  select: { id: true },
});
if (!store) {
  return NextResponse.json({ error: "Store not found" }, { status: 404 });
}
const since = new Date(Date.now() - DUPLICATE_WINDOW_MS);
// Atomic check-then-create: Serializable isolation ensures two concurrent
// POSTs can't both pass the "no recent duplicate" check and then both
// insert. The loser aborts with a serialization failure (P2034), which we
// translate to the same 429 the UI already handles.
try {
  const report = await prisma.$transaction(
    async (tx) => {
      const recent = await tx.storeReport.findFirst({
        where: {
          storeId,
          shopperId: session.user.id,
          createdAt: { gt: since },
        },
        select: { id: true },
      });
      if (recent) return null;

```

```

return tx.storeReport.create({
  data: {
    storeId,
    shopperId: session.user.id,
    type: type as ReportType,
    comment,
  },
  select: { id: true, type: true, comment: true, createdAt: true },
});
},
{ isolationLevel: "Serializable" },
);
if (!report) {
  return NextResponse.json(
    {
      error:
        "You've already reported this store in the last 24 hours. Thanks for helping keep info fresh.",
    },
    { status: 429 },
  );
}
return NextResponse.json({ report }, { status: 201 });
} catch (e) {
  if (
    typeof e === "object" &&
    e !== null &&
    "code" in e &&
    (e as { code?: string }).code === "P2034"
  ) {
    return NextResponse.json(
      {
        error:
          "You've already reported this store in the last 24 hours. Thanks for helping keep info fresh.",
      },
      { status: 429 },
    );
  }
  throw e;
}
}

```

?app/api/stores/[id]/reports/route.ts?

+39

Lines changed: 39 additions & 0 deletions

Original file line number Diff line number Diff line change

// Story 17: Owner-scoped listing of reports for their own store.

import { NextRequest, NextResponse } from "next/server";

import { prisma } from "@lib/prisma";

import { requireOwnerSession } from "@lib/require-owner-session";

export async function GET(

 _req: NextRequest,

 ctx: { params: Promise<{ id: string }> },

) {

 const gate = await requireOwnerSession();

 if (!gate.ok) return gate.response;


```

const { session } = gate;
const { id: storeId } = await ctx.params;
const store = await prisma.store.findUnique({
  where: { id: storeId },
  select: { id: true, ownerId: true },
});
if (!store) {
  return NextResponse.json({ error: "Store not found" }, { status: 404 });
}
if (store.ownerId !== session.user.id) {
  return NextResponse.json({ error: "Forbidden" }, { status: 403 });
}
const reports = await prisma.storeReport.findMany({
  where: { storeId },
  orderBy: { createdAt: "desc" },
  select: {
    id: true,
    type: true,
    comment: true,
    createdAt: true,
  },
  take: 100,
});
return NextResponse.json({ reports });
}

```

?components/Sidebar.tsx?

+1

Lines changed: 1 addition & 0 deletions

Original file line number Diff line number Diff line change

```

{ href: "/dashboard/profile", label: "Store Profile", icon: "?" },
{ href: "/dashboard/posts", label: "Fresh Updates", icon: "?" },
{ href: "/dashboard/deals", label: "Deals", icon: "?" },
{ href: "/dashboard/reports", label: "Reports", icon: "?" },
],
},
];

```

?components/StoreReportButton.tsx?

+175

Lines changed: 175 additions & 0 deletions

Original file line number Diff line number Diff line change

// Story 17: Shopper-facing "report incorrect info" button + lightweight form.

```

"use client";
import Link from "next/link";
import { useState } from "react";
import type { ViewerRole } from "@lib/viewer-role";
type ReportType =
  | "out_of_stock"
  | "incorrect_hours"
  | "wrong_price"
  | "other";
const TYPE_OPTIONS: { value: ReportType; label: string; hint: string }[] = [
  { value: "out_of_stock", label: "Out of stock", hint: "Item listed as in stock wasn't there" },
  { value: "incorrect_hours", label: "Hours wrong", hint: "Store hours look incorrect" },
  { value: "wrong_price", label: "Price wrong", hint: "Posted price didn't match in-store" },

```

```

{ value: "other", label: "Other", hint: "Anything else that seems off" },
];
type Props = {
  storeId: string;
  viewerRole: ViewerRole;
};
export default function StoreReportButton({ storeId, viewerRole }: Props) {
  const [open, setOpen] = useState(false);
  const [type, setType] = useState<ReportType>("out_of_stock");
  const [comment, setComment] = useState("");
  const [submitting, setSubmitting] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const [acknowledged, setAcknowledged] = useState(false);
  const callbackPath = `/stores/${storeId}`;
  const shopperLoginHref = `/shopper/login?callbackUrl=${encodeURIComponent(callbackPath)}`;
  if (viewerRole !== "shopper") {
    return (
      <div className="rounded-xl border border-dashed border-gray-200 bg-white px-4 py-3 text-[13px] text-
        <span>See something wrong with this page?</span>
        <Link
          href={shopperLoginHref}
          className="text-emerald-800 font-medium hover:underline"
        >
          Sign in as a shopper to report ?
        </Link>
      </div>
    );
  }
  async function submit() {
    setSubmitting(true);
    setError(null);
    try {
      const res = await fetch(`/api/stores/${storeId}/report`, {
        method: "POST",
        credentials: "include",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ type, comment: comment.trim() || undefined }),
      });
      if (!res.ok) {
        const data = await res.json().catch(() => ({}));
        throw new Error(data.error || "Could not submit report.");
      }
      setAcknowledged(true);
      setComment("");
    } catch (e) {
      setError(e instanceof Error ? e.message : "Could not submit report.");
    } finally {
      setSubmitting(false);
    }
  }
  if (acknowledged) {
    return (
      <div className="rounded-xl bg-emerald-50 border border-emerald-100 px-4 py-3 text-[13px] text-

```

gray-600

```
<span>Thanks ? your report was submitted.</span>
<button
  type="button"
  onClick={() => {
    setAcknowledged(false);
    setOpen(false);
  }}
  className="text-emerald-800 font-medium hover:underline"
>
Dismiss
</button>
</div>
);
}
if (!open) {
  return (
    <button
      type="button"
      onClick={() => setOpen(true)}
      className="inline-flex items-center gap-1.5 text-[12px] text-gray-500 hover:text-emerald-800
    >
    <span aria-hidden=?</span>
    Report incorrect info
  </button>
);
}
return (
  <div className="rounded-xl border border-gray-200 bg-white p-4 space-y-3">
    <div className="flex items-start justify-between gap-2">
      <div>
        <p className="text-[14px] font-semibold text-gray-800">
          Report incorrect info
        </p>
        <p className="text-[12px] text-gray-500 mt-0.5">
          Quick tip to help us keep the platform trustworthy.
        </p>
      </div>
      <button
        type="button"
        onClick={() => setOpen(false)}
        className="text-gray-400 hover:text-gray-600 text-[16px]"
        aria-label="Close report form"
      >
        ?
      </button>
    </div>
    <div className="flex flex-wrap gap-2">
      {TYPE_OPTIONS.map((opt) => (
        <button
          key={opt.value}
          type="button"
          title={opt.hint}
```

```

aria-pressed={type === opt.value}
onClick={() => setType(opt.value)}
className={`px-3 py-1.5 rounded-full text-[12px] font-semibold transition-colors ${
type === opt.value
? "bg-emerald-700 text-white"
: "bg-gray-50 text-gray-700 hover:bg-gray-100 border border-gray-200"
}`}
>
{opt.label}
</button>
)))
</div>
<textarea
maxLength={280}
rows={2}
value={comment}
onChange={(e) => setComment(e.target.value)}
placeholder="Optional ? what did you see? (280 chars max)"
className="w-full rounded-lg border border-gray-200 px-3 py-2 text-[13px] focus:outline-none
focus:ring-2 focus:ring-emerald-700"
/>
{error && (
<p className="text-[12px] text-red-600" role="alert">
{error}
</p>
)}
<div className="flex items-center justify-end gap-2">
<button
type="button"
onClick={() => setOpen(false)}
className="px-3 py-1.5 text-[13px] text-gray-600 hover:text-gray-800"
>
Cancel
</button>
<button
type="button"
disabled={submitting}
onClick={submit}
className="px-4 py-1.5 rounded-lg text-[13px] font-semibold bg-emerald-700 text-white hover:bg-emerald-800"
>
{submitting ? "Sending?" : "Send report"}
</button>
</div>
</div>
);
}
?prisma/migrations/20260419120000_store_reports/migration.sql?
+26
Lines changed: 26 additions & 0 deletions
Original file line number   Diff line number   Diff line change
-- CreateEnum
CREATE TYPE "StoreReportType" AS ENUM ('out_of_stock', 'incorrect_hours', 'wrong_price', 'other');
-- CreateTable

```

```

CREATE TABLE "store_reports" (
  "id" TEXT NOT NULL,
  "store_id" TEXT NOT NULL,
  "shopper_id" TEXT NOT NULL,
  "type" "StoreReportType" NOT NULL,
  "comment" TEXT,
  "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT "store_reports_pkey" PRIMARY KEY ("id")
);

-- CreateIndex
CREATE INDEX "store_reports_store_id_created_at_idx" ON "store_reports"("store_id", "created_at");
-- CreateIndex
CREATE INDEX "store_reports_shopper_id_store_id_created_at_idx" ON "store_reports"("shopper_id",

-- AddForeignKey
ALTER TABLE "store_reports" ADD CONSTRAINT "store_reports_store_id_fkey" FOREIGN KEY ("store_id")

-- AddForeignKey
ALTER TABLE "store_reports" ADD CONSTRAINT "store_reports_shopper_id_fkey" FOREIGN KEY

```

?prisma/schema.prisma?

+26

Lines changed: 26 additions & 0 deletions

Original file line number Diff line number Diff line change

store_new_deal

}

enum StoreReportType {

out_of_stock

incorrect_hours

wrong_price

other

}

model StoreOwner {

id String @id @default(uuid())

email String @unique

shopperNotifications ShopperNotification[]

profileViews StoreProfileView[]

itemSearches StoreItemSearch[]

reports StoreReport[]

@@map("stores")

}

alerts Alert[]

shopperNotifications ShopperNotification[]

reports StoreReport[]

@@map("shoppers")

}

@@map("auth_audit_logs")

}

/// Story 17: shopper-submitted reports of incorrect/outdated store info.

model StoreReport {

id String @id @default(uuid())

storeId String @map("store_id")

shopperId String @map("shopper_id")

type StoreReportType

```

comment String?
createdAt DateTime @default(now()) @map("created_at")
store Store @relation(fields: [storeId], references: [id], onDelete: Cascade)
shopper Shopper @relation(fields: [shopperId], references: [id], onDelete: Cascade)
@@index([storeId, createdAt])
@@index([shopperId, storeId, createdAt])
@@map("store_reports")
}

```

```

model Alert {

```

```

  id String @id @default(uuid())

```

```

  shopperId String @map("shopper_id")

```

```

?tests/StoreReportButton.test.tsx?

```

```

+121

```

```

Lines changed: 121 additions & 0 deletions

```

```

Original file line number   Diff line number   Diff line change

```

```

/**

```

```

 * Story 17 ? Report incorrect info (#121)

```

```

 *

```

```

 * Covers the shopper-facing report button:

```

```

 * - Non-shopper viewers see the sign-in CTA instead of the form.

```

```

 * - Shoppers can open the form, pick a type chip, and submit.

```

```

 * - Submit POSTs to /api/stores/:id/report with the selected type.

```

```

 * - Successful submit swaps in the "Thanks ? your report was submitted" ack.

```

```

 * - A 429 duplicate response surfaces the server's friendly error message.

```

```

 *

```

```

 * Tracking issue: #130

```

```

 */

```

```

import "@testing-library/jest-dom";

```

```

import { act, render, screen, waitFor } from "@testing-library/react";

```

```

import userEvent from "@testing-library/user-event";

```

```

import React from "react";

```

```

import StoreReportButton from "@components/StoreReportButton";

```

```

jest.mock("next/link", () => ({

```

```

  __esModule: true,

```

```

  default({ children, href }: { children: React.ReactNode; href: string }) {

```

```

    return <a href={href}>{children}</a>;

```

```

  },

```

```

}));

```

```

describe("StoreReportButton ? viewer role gating", () => {

```

```

  it("shows a sign-in CTA for logged-out viewers", () => {

```

```

    render(<StoreReportButton storeId="store-1" viewerRole={null} />);

```

```

    const link = screen.getByRole("link", { name: /sign in as a shopper/i });

```

```

    expect(link).toBeInTheDocument();

```

```

    expect(link.getAttribute("href")).toContain("/shopper/login");

```

```

    expect(link.getAttribute("href")).toContain("callbackUrl=");

```

```

  });

```

```

  it("shows a sign-in CTA for owner viewers (wrong account type)", () => {

```

```

    render(<StoreReportButton storeId="store-1" viewerRole="owner" />);

```

```

    expect(

```

```

      screen.getByRole("link", { name: /sign in as a shopper/i }),

```

```

    ).toBeInTheDocument();

```

```

  });

```

```

  it("renders the report trigger for shoppers", () => {

```

```

    render(<StoreReportButton storeId="store-1" viewerRole="shopper" />);

```

```

expect(
screen.getByRole("button", { name: /report incorrect info/i }),
).toBeInTheDocument();
});
});
describe("StoreReportButton ? shopper flow", () => {
afterEach(() => {
jest.restoreAllMocks();
});
it("submits the selected type and shows the acknowledgement", async () => {
const fetchMock = jest
.spyOn(global, "fetch")
.mockResolvedValue({
ok: true,
json: async () => ({ ok: true }),
} as Response);
render(<StoreReportButton storeId="store-42" viewerRole="shopper" />);
await userEvent.click(
screen.getByRole("button", { name: /report incorrect info/i }),
);
// Select a non-default chip to prove `type` flows through.
await userEvent.click(screen.getByRole("button", { name: /hours wrong/i }));
await act(async () => {
await userEvent.click(
screen.getByRole("button", { name: /send report/i }),
);
});
await waitFor(() => {
expect(fetchMock).toHaveBeenCalledWith(
"/api/stores/store-42/report",
expect.objectContaining({ method: "POST" }),
);
});
const body = JSON.parse(
(fetchMock.mock.calls[0][1] as RequestInit).body as string,
);
expect(body).toEqual({ type: "incorrect_hours" });
expect(
await screen.findByText(/thanks ? your report was submitted/i),
).toBeInTheDocument();
});
it("surfaces the server error on 429 dedupe", async () => {
jest.spyOn(global, "fetch").mockResolvedValue({
ok: false,
status: 429,
json: async () => ({
error: "You already reported this store recently. Try again later.",
}),
} as Response);
render(<StoreReportButton storeId="store-42" viewerRole="shopper" />);
await userEvent.click(
screen.getByRole("button", { name: /report incorrect info/i }),
);
await act(async () => {

```

```
await userEvent.click(
screen.getByRole("button", { name: /send report/i }),
);
});
expect(
await screen.findByText(/already reported this store recently/i),
).toBeInTheDocument();
// Still in form, not acknowledged.
expect(
screen.queryByText(/thanks ? your report was submitted/i),
).not.toBeInTheDocument();
});
});
```

the issue for tracking automation: US17: Dev Specs Automation #141

--- Turn 175 (assistant) ---

Creating the US17 dev spec from the codebase and merge metadata.

[REDACTED]

--- Turn 176 (assistant) ---

[REDACTED]

--- Turn 177 (assistant) ---

[REDACTED]

--- Turn 178 (assistant) ---

[REDACTED]

--- Turn 179 (assistant) ---

[REDACTED]

--- Turn 180 (assistant) ---

[REDACTED]

--- Turn 181 (assistant) ---

Here is what was added and how to open the PR.

New file

docs/dev-specs/US17-shopper-store-reports.md ?

same rub

Merge metadata: merge commit `d1d3c08` ? **2026-04-19T20:33:12Z` UTC** for PR

[#125](ht

Branch

- **dev-spec-us17`** (from current `main`, single commit adding this file only).

- Open PR: <https://github.com/gsha22/GrocerEase/pull/new/dev-spec-us17>

Suggested PR body:

``markdown

Summary

Adds a manual development specification for **US17** (shopper reports of incorrect store info),

matching

Links

Refs #121

Refs #141

File

`docs/dev-specs/US17-shopper-store-reports.md`

...

[REDACTED]

--- Turn 182 (user) ---

just curious, are you able to trigger the github action dev spec generation after a pr is

closed..a

--- Turn 183 (assistant) ---

Not with the workflow you have now.

In `.github/workflows/dev-spec-on-pr-approval.yml`, the `generate` job only runs when **all** of

these are

```yaml

github.event.pull\_request.state == 'open'

...

So it runs when someone submits an \*\*approved\*\* review while the PR is still \*\*open\*\* and targeting

\*\*`main`\*

Having the user story in the title only affects \*\*slug detection\*\* once the job actually runs; it

does not

To generate specs \*\*after\*\* merge you'd need a different trigger (for example `pull\_request` with

`types: [c

[REDACTED]

--- Turn 184 (user) ---

oh so you can adjust it? can you try it out

--- Turn 185 (assistant) ---

Reviewing the workflow to add a post-merge trigger while preserving the approval path.

[REDACTED]

--- Turn 186 (assistant) ---

Adding a `pull\_request` `closed` + `merged` trigger and updating the job condition. Fixing

`REVIEW

[REDACTED]

--- Turn 187 (assistant) ---

[REDACTED]

--- Turn 188 (assistant) ---

[REDACTED]

--- Turn 189 (assistant) ---

[REDACTED]

--- Turn 190 (assistant) ---

[REDACTED]

--- Turn 191 (assistant) ---

[REDACTED]

--- Turn 192 (assistant) ---

Here is what changed and what ?try it out? means here.

### What we changed

- 1. **Triggers** (`.github/workflows/dev-spec-on-pr-approval.yml`):
  - **pull\_request\_review** (unchanged): approve a PR into **main** while it is still **open**.
  - **pull\_request** with **types: [closed]**: same slug rules, but only when the PR **merged** ==
- 2. **Job if**: Runs when base is **main** and either the **approval + open** path or the **closed +**
- 3. **REVIEW\_SUBMITTED\_AT**: Set only for **pull\_request\_review** so the merge event does not
- 4. **merge\_context.md**: Adds **GITHUB\_EVENT\_NAME** and labels the approval timestamp as **n/a**
- 5. **Commit / docs PR text**: Uses **merge** vs **approval** based on **GITHUB\_EVENT\_NAME**.
- 6. **docs/dev-specs/README.md**: Documents both triggers and that **approve then merge** can cause

Workflow display name is now **?Dev specification (on PR approval or merge)?**.

### ?Try it out? from here

Actions only run what?s on the **default branch** (or whatever your repo rules are) once this YAML

The workflow + README updates are **committed and pushed** on **dev-spec-us17** (commit

If you want the CI change **separate** from the US17 doc PR, cherry-pick **5b2a09a** onto a branch

[REDACTED]

--- Turn 193 (user) ---

can u generate a pdf chatlog from when i told you to implement the dev spec automation

--- Turn 194 (assistant) ---

Searching agent transcripts for the dev-spec automation conversation and attempting to produce a

[REDACTED]